# Universal Channel Rebalancing: Flexible Coin Shifting in Payment Channel Networks

Stefan Dziembowski
University of Warsaw
stefan.dziembowski@crypto.edu.pl

Shahriar Ebrahimi
Zero Savvy
sh.ebrahimi92@gmail.com

Omkar Gavhane
Samsung R&D Institute India - Bangalore
omkar.g.1998@gmail.com

Susil Kumar Mohanty*
University of Warsaw
s.mohanty@uw.edu.pl

## Abstract

Payment Channel Networks (PCNs) enhance blockchain scalability by enabling off-chain transactions. However, repeated unidirectional multi-hop payments often cause channel imbalance or depletion, limiting scalability and usability. Existing rebalancing protocols, such as Horcrux [NDSS'25] and Shaduf [NDSS'22], rely on on-chain operations, which hinders efficiency and broad applicability. We propose Universal Channel Rebalancing (UCRb), a blockchain-agnostic, fully off-chain framework that ensures correct behavior among untrusted participants without on-chain interaction. UCRb incorporates the following core innovations: (1) a fair and reliable incentive-compatible mechanism that encourages voluntary user participation in off-chain channel rebalancing, (2) integration of Pedersen commitments to achieve atomic off-chain payments and rebalancing operations, while ensuring balance security, and (3) zero-knowledge proofs to enable privacy-preserving channel initialization and coin shifting, ensuring that user identities and fund allocations remain hidden throughout the process.

We evaluate UCRb using real-world Lightning Network dataset and compare its performance against state-of-the-art solutions including Horcrux, Shaduf, and Revive [CCS'17]. UCRb exhibits a success ratio enhancement between 15% and 50%, while also reducing the required user deposits by 72%–92%. It maintains an almost negligible rate of channel depletion. Additionally, the long-term performance of UCRb is roughly 1.5 times that of its short-term performance, suggesting that continuous operation leads to improved efficiency. We implement a prototype for UCRb smart contracts and demonstrate its practicality through extensive evaluation. As `CoinShift` operations require no on-chain interaction, the protocol incurs minimal gas costs. For instance, opening and closing channels with 10 neighbors costs only 130K~160K gas—significantly lower than comparable solutions.

## 1 Introduction

Permissionless blockchains, such as Bitcoin [21] and Ethereum [30], are significantly reshaping the financial landscape through decentralized consensus mechanisms like Proof of Work (PoW) and Proof of Stake (PoS). These consensus techniques help establish and maintain a secure, distributed ledger that records all transactions. However, despite their numerous advantages, a well-known challenge of these blockchains is their limited scalability, which results in low transaction throughput, high fees, and increased latency. These issues hinder the widespread adoption of blockchain solutions. For example, on April 30, 2024, the highest number of daily Bitcoin transactions reached $927,0101$[1], while Ethereum peaked at 1.961 million[2] transactions on January 14, 2024. Nevertheless, Bitcoin and Ethereum can only process about 7 and 15 transactions per second (TPS), respectively. In contrast, traditional payment systems like Visa can handle more than $65,000$ TPS[3]. This significant gap in transaction capacity is primarily due to the permissionless nature of these blockchains and the decentralized consensus mechanisms, which inherently limit the speed and scalability of the network.

Payment Channels (PCs) [3, 25] have emerged as a promising Layer 2 solution to address the scalability limitations of blockchain systems. By locking funds in a multi-signature smart contract, two users can establish a PC that enables them to conduct multiple off-chain transactions by simply updating the channel state. The channel can be closed at any time by broadcasting the final state to the blockchain, thereby reducing on-chain interactions and transaction costs. However, creating a dedicated payment channel between every pair of users is economically inefficient, as each party must lock capital in every individual channel. To overcome this challenge, Payment Channel Networks (PCNs) [15, 16] have been proposed. PCNs enable value transfers between users who do not share a direct channel by routing payments through intermediary nodes using existing channels. A prominent example is Bitcoin's Lightning Network (LN), which currently supports over $322 million USD in locked value, spanning more than 14,000 nodes and 64,000 channels[4]. While PCNs significantly enhance blockchain scalability, they still face several open challenges, particularly with respect to security, privacy, and routing efficiency [18, 19, 29]. Among the most critical issues is *channel depletion*, wherein repeated unidirectional transactions along a specific path drain liquidity from one side of a channel. This imbalance prevents further payments in the same direction, leading to transaction failures, inefficient rerouting, or fallback to costly on-chain settlements. Such limitations undermine the usability and throughput of PCNs, and call for more robust, liquidity-aware routing and rebalancing mechanisms.

Several strategies have been proposed to address the persistent problem of channel depletion in PCNs. A basic approach involves manually refunding a depleted channel by closing and reopening it, which requires two on-chain transactions. While straightforward,

*Corresponding author

[1] https://ycharts.com/indicators/bitcoin_transactions_per_day
[2] https://ycharts.com/indicators/ethereum_transactions_per_day
[3] https://usa.visa.com/solutions/crypto/deep-dive-on-solana.html
[4] https://1ml.com/statistics

this process is costly and introduces delays. More efficient alternatives, such as Splicing [1] and LOOP [2], reduce this to a single on-chain transaction. Nevertheless, these solutions still depend on blockchain interaction and require users to possess sufficient on-chain liquidity. There have been many other approaches to rebalance funds across adjacent channels, including Revive [14], HIDE & SEEK [5], Thora [4], and Wiser [28]. These protocols perform multi-party off-chain rebalancing through coordinated circular payments, significantly reducing the burden on the blockchain. However, their effectiveness is constrained by several practical limitations: participants must be able to form directed cycles that align with the desired flow of funds; a trusted coordinator is needed to gather rebalancing requests, identify appropriate cycles, and facilitate transactions; and cooperation among all cycle participants is essential. These requirements limit their applicability in dynamic and decentralized environments.

To overcome these barriers, Shaduf [12] proposes a non-cyclic rebalancing approach by binding two adjacent channels via an on-chain contract, enabling off-chain fund transfers between them. While this model minimizes path restrictions and supports repeated rebalancing operations, it is hindered by high setup costs, on-chain dependency, and limited scalability due to its focus on single-node coordination. Horcrux [27] presents an improved off-chain rebalancing framework by using virtual channel [9] to maintain liquidity symmetry and enhance payment reliability. It achieves this by reducing reliance on complex scripting and ensuring transaction success when intermediaries maintain sufficient adjacent-channel balances. However, Horcrux requires on-chain operations for both setup and closure, introducing overhead that limits its off-chain efficiency. Its rebalancing mechanism is further constrained by locking coins only between adjacent intermediaries along the payment path, excluding non-participating users from contributing to the rebalancing process. This design significantly limits flexibility and scalability, and fails to achieve optimal rebalancing in scenarios where broader liquidity coordination is essential for maintaining high payment success rates. Most recently, Flexichain [20] offers an alternative model that enables users to distribute and share liquidity across multiple channels by maintaining a user-level fund pool instead of locking assets in individual channels. This enhances flexibility and improves payment success rates. Nevertheless, Flexichain relies heavily on neighbor cooperation without offering explicit incentives, making participation voluntary and fragile. In adversarial cases, dispute resolution requires reverting to on-chain arbitration, which significantly increases delay and incurs storage overhead.

Despite these advancements, the problem of efficient, incentive-compatible, and scalable off-chain channel rebalancing remains unresolved. The comparison between the existing work and the proposed universal channel rebalancing (UCRb) is presented in Table 1. Existing solutions either suffer from high on-chain overhead, complex coordination requirements, or limited flexibility in dynamic PCNs. These challenges in PCNs underscore the need for a robust, practical solution to channel depletion—without compromising performance, privacy, security, or decentralization, raising a fundamental research question:

*"Can we design a fully off-chain, blockchain-agnostic, and privacy-preserving channel rebalancing protocol that relies solely on digital signatures in a non-cooperative setting, and enables instant settlement without blockchain interaction except in case of disputes?"*

**Table 1: The comparisons among the state-of-the-art works**

| | [14] | [4] | [12] | [27] | [20] | Prop. |
|---|---|---|---|---|---|---|
| **No On-chain setup** | ✓ | ✓ | ✗ | ✗ | ✓ | ✓ |
| **Sustainability** | ✗ | ✓ | ✗ | ✓ | ✓ | ✓ |
| **Acyclic Construction** | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| **Reliability** | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ |
| **Flexibility** | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ |
| **Incentive** | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ |
| **Instant Finality** | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| **Compatible** | ETH | BTC | ETH | BTC | ETH | GEN |

- [14]: Revive, [4]: Thora, [12]: Shaduf, [27]: Horcrux, [20]: Flexichain, ETH: Ethereum, BTC: Bitcoin, GEN: Generic

In this work, we answer this question affirmatively by proposing Universal Channel Rebalancing (UCRb), a novel, highly scalable, and blockchain-agnostic protocol that addresses the limitations of existing rebalancing solutions in PCNs. UCRb enables efficient, privacy-preserving, and fully off-chain channel rebalancing without requiring cycle formation, trusted coordination, or smart contract support. It relies solely on standard digital signatures (e.g., Schnorr [26], ECDSA [13], or BLS [7]), and requires only minimal communication among participating users. This design makes UCRb both lightweight and platform-independent. Moreover, UCRb incorporates incentive-compatible mechanisms that encourage voluntary participation, even in decentralized and potentially adversarial environments, while fallback to on-chain resolution is required only in rare cases of dispute. The key contributions of this work are as follows:

- **Universal Channel Rebalancing:** We introduce UCRb, a new construction for PCNs that enables adaptive, flexible, and highly efficient off-chain channel rebalancing, instantly shift the coins along the payment path. Unlike existing approaches that require two on-chain transactions per channel: one for opening and one for closing. UCRb reduces dramatically this overhead by allowing each user to interact with the blockchain only twice: once to establish channels with a set of neighbors through a single deposit, and once to settle all balances upon closure.
- **Trustless Incentive Mechanism:** UCRb introduces a robust and fair, incentive-compatible design that ensures reliable off-chain rebalancing and motivates honest participation even in adversarial and non-cooperative environments, promoting voluntary rebalancing, and maintains efficiency.
- **Balance Security:** UCRb introduces PCTLC, a Pedersen Commitment [24] based Time-Lock Contract that ensures the atomic execution of both Payment and CoinShift operations. This mechanism guarantees secure fund transfers without loss or duplication, thereby preserving balance security.
- **Privacy Enhancement:** UCRb guarantees privacy by hiding both the internal coin allocations and the rebalancing process. During the CoinDeposit operation, the user proves in ZK that their funds have been correctly partitioned between neighbors, without revealing the specific amounts. Similarly, in the CoinShift

operation, UCRb ensures that the identities of the participating neighbors remain hidden, while still proving that the `CoinShift` is valid and consistent with rules.

- **Universal Efficiency:** UCRb is designed to be lightweight and broadly compatible, relying solely on standard digital signature verification (e.g., Schnorr [26], ECDSA [13], or BLS [7]) without requiring platform-specific scripting or smart contract functionality. This ensures practical deployment across a wide range of blockchain systems while maintaining an efficient construction.
- **Implementation and Evaluation:** UCRb distinguishes itself from prior work by eliminating the need for on-chain transactions during the CoinShift operations. As a result, it achieves significantly lower gas costs compared to existing solutions such as Shaduf [12] and Horcrux [27]. Users only interact with the blockchain during the initial coin deposit and the final coin settlement phases. For example, the total gas cost for opening and closing all channels with 10 neighbors is approximately 130K∼160K gas, which is substantially lower than the cumulative costs incurred by competing approaches. We present a proof-of-concept implementation of UCRb, Our performance evaluation includes network depletion as a key metric, and we assess both short-term and long-term behavior by executing 20 and 200 batches of 50,000 random transactions, respectively. Compared to existing schemes [12, 14, 27], UCRb achieves a 15%-50% improvement in payment success ratio, a 72%-92% reduction in user deposit requirements, and exhibits a 1.5x performance gain in extended operation. Notably, UCRb sustains an almost negligible channel depletion rate throughout, outperforming prior approaches in this aspect. Furthermore, it ensures a fair distribution of incentives among all nodes participating in the coin shift operations for each transaction.

## 2 PRELIMINARIES AND BACKGROUND

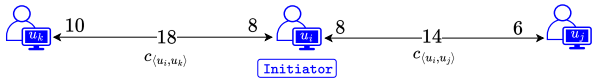
We summarize the key preliminaries and cryptographic tools relevant to our proposed work are provided in Appendix A.

## 3 Universal Channel Rebalancing Overview

### 3.1 Overview

At the core of off-chain payment networks lies the concept of a *payment channel* [15, 16]. Unlike traditional systems, which require separate on-chain transactions to open each channel [25], thereby tightly coupling channel operations with the blockchain, our model adopts a distinct and partially decentralized approach. When a user wishes to establish payment channels with neighboring participants, the process begins with an off-chain *channel opening request* sent to all intended neighbors. Upon receiving consent from each neighbor, involved parties collaboratively generate a *multisignature* that defines the agreement for channels. The user then deposits funds along with the multisignature in a single on-chain transaction. The transaction simultaneously opens multiple payment channels, thereby significantly reducing on-chain overhead.

Consider Example (a) below, where user $u_i$ initially sends a *channel opening request* to neighbors $u_j$ and $u_k$. Upon receiving consent from them, $u_i$ deposits (or locks) a total of 16 coins along with the multisignature on the blockchain to enable subsequent off-chain operations. Following this, user $u_i$ established two separate payment channels: channel $c_{\langle u_i,u_j\rangle}$ with $u_j$, and channel $c_{\langle u_i,u_k\rangle}$ with $u_k$. The deposited coins are then equally allocated across the two channels, assigning 8 coins to each.

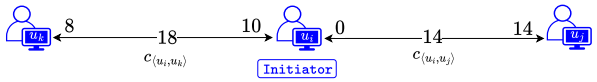



(a) **Coin allocation by user $u_i$ across channels $c_{\langle u_i,u_j\rangle}$ and $c_{\langle u_i,u_k\rangle}$**

When allocating coins across channels, it is important to uphold *balance security* [15], which safeguards against *double-allocation attacks*, where a user illegitimately assigns the same coin to multiple channels, potentially resulting in financial losses for honest participants. To prevent such inconsistencies, strict safeguards must be imposed to ensure that coins are allocated only once to a channel, and that allocations at the destination channels are independently verifiable. Maintaining this integrity requires that every coin allocation operation be mutually approved by all neighboring users involved in the associated channels. Moreover, for off-chain allocations to be effective and enforceable, they must remain valid for on-chain settlement at the time of claim or redemption.

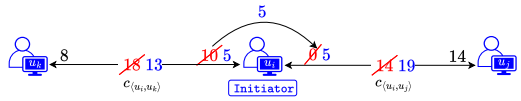
Now that the network is established, it is ready to perform off-chain payment operations. Any two users can initiate payments either through directly connected channels or via multi-hop connections. Multi-hop payment system rely on Hash Time-Locked Contracts (HTLCs) [25] to guarantee atomicity, such mechanisms are known to be vulnerable to *wormhole attacks* [15]. Rather than employing the HTLC approach directly, our protocol adopts a fundamentally different locking and unlocking mechanism, preserving *balance security* [15], ensuring that intermediate users are protected from financial loss, even in the presence of malicious participants.

Suppose user $u_i$ transfers 6 coins directly to $u_j$, followed by a transfer of 2 coins from $u_k$ to $u_j$ via $u_i$. After these payment operations, $u_i$ holds a balance of 0 coins in $c_{\langle u_i,u_j\rangle}$ and 10 coins in $c_{\langle u_i,u_k\rangle}$, as depicted in example $(b)$ below. Due to the depleted balance in $c_{\langle u_i,u_j\rangle}$, $u_i$ is temporarily unable to initiate payments through that channel. However, under the universal channel rebalancing (UCRb) approach, $u_i$ can revive the usability of $c_{\langle u_i,u_j\rangle}$ by shifting coins from neighboring channel $c_{\langle u_i,u_k\rangle}$. This coin shift is performed entirely off-chain, without requiring any blockchain interaction.




(b) **Channel depletion occurs at user $u_i$ in channel $c_{\langle u_i,u_j\rangle}$**

As illustrated in Example $(c)$ below, user $u_i$ performs coin shift operations by transferring 5 coins from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$. This coin shift enables $u_i$ to resume initiating payments through $c_{\langle u_i,u_j\rangle}$. The coin shift is executed entirely off-chain, involving only local balance adjustments across $u_i$'s own channels, without any blockchain interaction. Moreover, this mechanism can be generalized to support multiple coin shift operations between any neighboring channels in arbitrary directions, offering a flexible and adaptive rebalancing strategy.



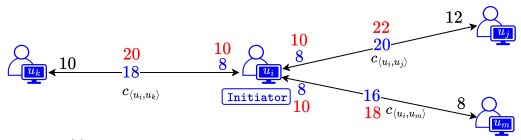

(c) Coin shift performed by user $u_i$, shifting 5 coins from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$

As part of the coin shift process, it is important to ensure *balance security* [15], also known as protection against *double-shifting attacks* [12, 27], where honest users might otherwise suffer financial losses. In order to prevent these scenarios, safeguards must be in place to ensure that coins can only be transferred to another channel if they are available for withdrawal from one channel and simultaneously accepted into another channel for future payments. Each coin shift operation should be mutually approved by the counterparties involved, namely, the two users participating in the source and destination channels. Moreover, to ensure that off-chain rebalancing reflects valid and enforceable state transitions, the shifted coins must be valid for on-chain settlement at the point of claim or redemption of the coins.

At a certain point, user $u_i$ may wish to settle the balances and close all active payment channels with neighboring users. To initiate this process, $u_i$ shares the latest updated state with each of its channel counterparties $u_j$ and $u_k$. Each neighbor independently verifies the correctness of the received state and, upon validation, provides their consent. Once all consents are received, $u_i$ aggregates them into a multisignature on the final state and submits this to the blockchain. The blockchain then verifies both the validity of the multisignature and the consistency of the updated state. If all verifications pass, the blockchain proceeds to settle the deposits accordingly and finalizes the coins.

## 3.2 Necessity of CoinAllocation Proof

Allocating coins across neighboring channels is an effective strategy for initializing and maintaining the operability of payment channels. However, to ensure the validity and acceptance of these allocations, each coin allocation must be broadcast to all neighboring users. This transparency is crucial for coordination and mutual verification. Witout it, the system becomes susceptible to *double-allocation attacks*, where a user could illegitimately assign the same coin to multiple channels, thereby compromising the integrity and security of the payment network.

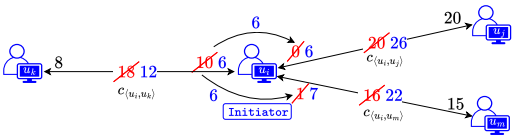



(d) Coin allocation by user $u_i$ across channels $c_{\langle u_i,u_j\rangle}$, $c_{\langle u_i,u_k\rangle}$, and $c_{\langle u_i,u_m\rangle}$

Consider Example (d) above, where user $u_i$ establishes three payment channels $c_{\langle u_i,u_j\rangle}$, $c_{\langle u_i,u_k\rangle}$, and $c_{\langle u_i,u_m\rangle}$ with users $u_j$, $u_k$, and $u_m$, respectively. Initially, all three channels are created with zero balance. Suppose $u_i$ deposits 24 coins on-chain and intends to allocate them equally, assigning 8 coins to each channel. However, acting maliciously, $u_i$ falsely attempts to allocate 10 coins to each of the three channels. Instead of broadcasting this allocation to all neighboring users, $u_i$ privately communicates the allocation to each individual counterparty $u_j$, $u_k$, and $u_m$ separately. Through this deceptive strategy, $u_i$ effectively assigns overlapping coin values across multiple channels, misrepresenting its actual on-chain holdings and undermining the fairness and correctness of the system. From the perspective of each individual neighbor, the allocation appears valid and independent, leading the honest users $u_j$, $u_k$, and $u_m$ to accept their respective allocations as legitimate. In reality, the total claimed allocation sums to 30 coins, exceeding $u_i$'s actual deposit of 24 coins, resulting in an overcommitment of 6 coins. This discrepancy leads to potential financial loss for one or more honest users, highlighting the critical need for global coordination and verification in coin allocation mechanisms. A detailed description of the coin allocation operation is provided in Section 5.

## 3.3 Necessity of CoinShift Proof

Coin shifting from well-funded to depleted channels effectively mitigates balance skewness and prevents channel depletion. However, without mutual verification among participants, such operations are vulnerable to *double-shifting attacks* [12, 27], where the same coins are fraudulently reassigned across multiple channels.




(e) Coin shift performed by user $u_i$, shifting 6 coins from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_m\rangle}$ or channel $c_{\langle u_i,u_j\rangle}$

Consider Example (e), where user $u_i$ maintains three channels: $c_{\langle u_i,u_j\rangle}$, $c_{\langle u_i,u_k\rangle}$, and $c_{\langle u_i,u_m\rangle}$, with balances of 0, 10, and 1 coins respectively. Suppose $u_i$ tries to shift 6 coins from $c_{\langle u_i,u_k\rangle}$ to both $c_{\langle u_i,u_j\rangle}$ and $c_{\langle u_i,u_m\rangle}$. While both receivers independently perceive the shift as valid, the combined commitment of 12 coins exceeds $u_i$'s capacity, resulting in overcommitment and possible financial loss. To prevent this, all coin shifts must be transparently communicated and confirmed through synchronous off-chain message exchanges among the involved users. Without such coordination, malicious users can exploit the opacity of individual channel balances to execute inconsistent shifts. Unlike Shaduf and Horcrux, which rely on on-chain mechanisms to mitigate these issues, our protocol maintains full off-chain operation. It enforces a *coin-shift agreement* among all participants (e.g., $\{u_k, u_i, u_j\}$ or $\{u_k, u_i, u_m\}$), ensuring that only one shift can be finalized at a time, thereby eliminating the risk of double-shifting. The detailed coin shift protocol is described in Section 5.

## 3.4 Necessity of Lock/ Unlock Commitments

The primary benefit of executing multi-hop payment operations off-chain lies in the significant reduction of on-chain costs, thereby enhancing the scalability and efficiency of the network. However, ensuring the security of such multi-hop off-chain payments necessitates careful protocol design. Without appropriate safeguards, malicious users may collude to exploit protocol vulnerabilities and steal the processing fees intended for honest intermediaries. This type of exploit is known as the *wormhole attacks* [15, 16]. To mitigate this risk and simultaneously ensure atomicity in multi-hop payments across the payment channel network, we introduce a

locking and unlocking mechanism based on the Pedersen commitment scheme [24]. This design ensures that honest intermediaries are protected from financial losses, even in the presence of adversarial participants. A detailed description of the commitment-based locking and unlocking mechanism for secure multi-hop payments is provided in Section 5.

### 3.5 Necessity of CoinSettlement Confirmation

When a user intends to settle its coins on-chain, it must first share the most recent updated state with each of its channel neighbors. The settlement process cannot proceed until the user receives explicit consent from all involved neighbors. Prematurely submitting the state to the blockchain without mutual agreement may result in state inconsistencies and lead to disputes regarding the correctness of the submitted state. Resolving such disputes often requires additional on-chain operations, which are both costly and time-consuming. To prevent these issues, the user generates a settlement agreement in the form of a multisignature, which aggregates the consent of all neighboring users. The updated state, along with the corresponding multisignature, is then submitted to the blockchain. This approach ensures that the state is consistent and verifiable, allowing the blockchain to settle the coins correctly and securely.

### 3.6 Incentive Fee Model based on CoinShift

In off-chain payment networks, users behave rationally and non-cooperatively, engaging only when it serves their self-interest. This often leads to liquidity imbalances, causing channel depletion and failed transactions that may require costly on-chain settlements. To address this, **UCRb** introduces an incentive-compatible mechanism that rewards users for contributing to channel rebalancing via CoinShift operations. When a user shifts coins to rebalance a depleted channel, it earns incentives from other nodes along the payment path. These incentives are proportional to the user's rebalancing effort, measured by the number and volume of coin shifts, and are partially redistributed to neighboring nodes that contributed liquidity. We propose two general models for incentive distribution:

- **Case 1 (Sender-Funded Model):** The sender locks an extra amount, separate from routing fees, to cover rebalancing costs. Each intermediate node locks and later computes its incentive share based on contributions. These computations are verified by successors and finalized by the sender during the unlocking phase.
- **Case 2 (Relay-Funded Model):** Intermediate nodes lock a small rebalancing amount. During unlocking, each node pays its successor based on their own and their predecessor's contributions, and then claims the corresponding amount from the predecessor. Each node verifies the correctness of its successor's calculations to ensure fair distribution.

These mechanisms ensure that no participant incurs a loss and that compensation aligns with each user's role. Rational users are thus motivated to participate in rebalancing, not out of altruism, but through self-interest. This promotes long-term balance and operability of the network. Full technical details are provided in Section 5.

For a more discussion and additional insights on this topic, we refer the reader to Appendix C.

## 4 Formal Definitions

### 4.1 Notation

We now present the notations used to formally define the UCRb and its operations.

*4.1.1 Coin Deposit.* We denote the blockchain as $\mathcal{B}$, where each user or node is represented by $u_i$. The set of neighboring users connected to $u_i$ is denoted by $\mathcal{N}(u_i)$, with each neighbor identified as $u_j \in \mathcal{N}(u_i)$. Each user maintains a blockchain wallet with an associated on-chain balance $\mathcal{B}[u_i]$. The off-chain balance or collateral locked by user $u_i$ is denoted by $v_{u_i}$, which remains locked for a specified duration $t_{u_i}$. For off-chain operations, each user defines a payment processing (relay) fee $f_{u_i}$, a lower rebalancing fee rate $l_{u_i}$, and a higher rebalancing fee rate $h_{u_i}$. The payment channel between users $u_i$ and $u_j$ is identified by $c_{\langle u_i,u_j\rangle}$. The directional channel balances are denoted as $\Upsilon_{ij}$ (from $u_i$ to $u_j$) and $\Upsilon_{ji}$ (from $u_j$ to $u_i$), while the total channel capacity is represented as $\complement_{ij}$ or equivalently $\complement_{ji}$, which is $\complement_{ij} = \Upsilon_{ij} + \Upsilon_{ji}$. The digital signature generated by user $u_i$ is denoted as $\sigma_{u_i}$, while a multisignature is represented by $\sigma$. The list of currently open channels is denoted by $OC$. We denote a positive acknowledgment (confirmation) by $\top$, and a negative acknowledgment (rejection or failure) by $\bot$.

*4.1.2 Coin Allocation.* The notation $\mathsf{P}_{\mathsf{NIZK}}$ refers to the prover algorithm for a non-interactive zero-knowledge (NIZK) proof, while $\mathsf{V}_{\mathsf{NIZK}}$ denotes the corresponding verifier algorithm. The coin allocation proof generated by user $u_i$ is denoted by $\pi^{ca}_{u_i}$, and the corresponding set of allocated coins is represented by $\Upsilon$.

*4.1.3 Payment.* A payment path, or PCN, is represented as $\mathcal{P} : \{u_0 \rightarrow \cdots \rightarrow u_i \rightarrow \cdots \rightarrow u_n\}$, where $u_0$ denotes the payer (sender), $u_i$ represents intermediate users (relay nodes), and $u_n$ is the payee (receiver). The value $v$ indicates the off-chain payment amount, while $\delta$ denotes the additional amount reserved for channel rebalancing. For a channel $c_{\langle u_i,u_j\rangle}$, the value $v_{ij}$ corresponds to the amount locked by $u_i$ for a duration $t_{ij}$, and $v_{ji}$ represents the amount claimed by $u_j$ during the unlock phase. The parameters $\alpha_{ij}$, $\beta_{ij}$, $\gamma_{ij}$, $\varphi$, and $\vartheta_{ih}$ denote locking and unlocking paramters computed by $u_i$ for the same channel. The digital signature $\sigma_{ij}$ is generated by $u_i$ for $c_{\langle u_i,u_j\rangle}$, whereas $\sigma_{ji}$ is generated by $u_j$ for the same channel. The set of coin shift proofs is denoted by $\pi^{cs}_{ij}$, with the corresponding public input set represented as $\mathrm{pi}_{ij}$ for the channel $c_{\langle u_i,u_j\rangle}$.

*4.1.4 Coin Shift.* We denote $v_{ij}$ as the total amount of coins that user $u_i$ intends to shift or rebalance in the payment channel $c_{\langle u_i,u_j\rangle}$. Similarly, $v^i_{kj}$ represents the specific amount of coins that user $u_i$ shifts from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$. To represent the generation of a pseudo-random element $x$ from a prime-order group $\mathbb{Z}_p$, we use the notation $x \xleftarrow{\$} \mathbb{Z}_p$. In the context of coin shift operations, we adopt the prime notation (also referred to as dashed or $'$) to indicate updated parameters. The locking parameters generated by user $u_i$ for channel $c_{\langle u_i,u_k\rangle}$ are denoted as $\alpha'_{ik}$, $\beta'_{ik}$, $\gamma'_{ik}$, $\vartheta'_{ik}$, and $\ell'_{ik}$. The signature generated by $u_i$ for channel $c_{\langle u_i,u_k\rangle}$ is represented as $\sigma^i_{ik}$. Additionally, the updated channel balance and channel capacity are denoted by $\Upsilon'_{ik}$ and $\complement'_{ik}$, respectively. The

coin shift proof is represented as $\pi^i_{kj}$, with its corresponding public input given by $\text{pi}^i_{kj}$, and the associated statement denoted as stmt. This proof is generated by user $u_i$ to attest to the validity of the coin shift from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$. The unique coin shift number is denoted as cs. Additionally, the coin shift list is represented by $\mathcal{CS}$.

*4.1.5 Coin Settlement.* The notation $\mathsf{F}_{\text{state}}$ represents the final state of all channels connected to user $u_i$. The current state of an individual channel $c_{\langle u_i,u_j\rangle}$ is denoted by $\text{state}(c_{\langle u_i,u_j\rangle})$. Additionally, the set of currently closed channels is represented by $\mathcal{CC}$.

## 4.2 Definition

We model Universal Channel Rebalancing (UCRb) as a bi-directed and weighted graph $\mathbb{G} := (\mathbb{V}, \mathbb{E}, \Omega)$, where $\mathbb{V}$ denotes the set of blockchain users with a weight function $w$, $w : \mathbb{V}\times\mathbb{V} \to \mathbb{R}^+$ denotes the collateral between nodes, $\mathbb{E} \subseteq \mathbb{V} \times \mathbb{V}$ denotes the set of active payment channels between two user. An UCRb is defined with respect to a blockchain $\mathcal{B}$ and is equipped with the five operations described below:

- $0/1 \leftarrow \text{coinDeposit}(u_i, \mathcal{N}(u_i), v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, t_{u_i}, \sigma)$. On input parameters $\mathcal{N}(u_i)$, $v_{u_i}$, $f_{u_i}$, $l_{u_i}$, $h_{u_i}$, and $t_{u_i}$, a blockchain user $u_i \in \mathbb{V}$ invokes the operation coinDeposit. For each neighbor $u_j \in \mathcal{N}(u_i)$, the operation initiates the creation of a new payment channel $c_{\langle u_i,u_j\rangle} \in \mathbb{E}$. Upon successful operation, it broadcasts the tuple $(u_i, \mathcal{N}(u_i), v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, \sigma)$ to the blockchain $\mathcal{B}$, and it returns 1. If the operation fails, it returns 0.
- $0/1 \leftarrow \text{coinAllocation}(u_i, \mathcal{N}(u_i), v_{u_i})$. On input parameters $\mathcal{N}(u_i)$ and $v_{u_i}$, an user $u_i \in \mathbb{V}$ invokes the operation coinAllocation. For each neighbor $u_j \in \mathcal{N}(u_i)$, it allocates coins $\Upsilon_{ij}$ and $\Upsilon_{ji}$, and generates a coin allocation proof $\pi^{ca}_{u_i}$. Then, it sends the tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i})$ to $u_j$. Upon successful operation, it returns 1. Otherwise, it returns 0.
- $0/1 \leftarrow \text{payment}((c_{\langle u_0,u_1\rangle}, \cdots, c_{\langle u_{n-1},u_n\rangle}), v)$. On input a list of payment channels $(c_{\langle u_0,u_1\rangle}, \ldots, c_{\langle u_{n-1},u_n\rangle})$ and a payment amount $v$, the payer $u_0$ invokes the operation payment. The operation first verifies whether the given channels form a valid payment path from the payer $u_0$ to the payee $u_n$, and checks that each channel $c_{\langle u_i,u_{i+1}\rangle}$ along the path has a current balance $\Upsilon_{\langle u_i,u_{i+1}\rangle} \geq v_{\langle u_i,u_{i+1}\rangle}$, where $v_{\langle i,i+1\rangle} = v_{01} - \sum_{j=1}^{i} f_{u_j}$. If all conditions are satisfied, the operation deducts $v_{\langle i,i+1\rangle}$ from the balance of each respective channel $c_{\langle u_i,u_{i+1}\rangle}$ and returns 1. Otherwise, no changes are made to any channel balance and the operation returns 0.
- $0/1 \leftarrow \text{coinShift}(u_i, c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj})$. On input input parameters $c_{\langle u_i,u_k\rangle}$, $c_{\langle u_i,u_j\rangle}$, and $v^i_{kj}$, a user $u_i \in \mathbb{V}$ invokes the operation coinShift. This operation transfers an amount $v^i_{kj}$ of coins from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$, and generates a corresponding coin shift proof $\pi^i_{kj}$. Upon successful execution, it returns 1; otherwise, it returns 0.
- $0/1 \leftarrow \text{coinSettlement}(u_i, \mathsf{F}_{\text{state}}, \sigma)$. On input a list of final channel states $\mathsf{F}_{\text{state}}$, a blockchain user $u_i \in \mathbb{V}$ invokes the operation coinSettlement. It sends $\mathsf{F}_{\text{state}}$ to each neighbor $u_j \in \mathcal{N}(u_i)$, along with $u_i$'s signature $\sigma_{u_i}$. Upon successful operation, it broadcasts the tuple $(u_i, \mathsf{F}_{\text{state}}, \sigma)$ to the blockchain $\mathcal{B}$, and the operation returns 1. If the settlement fails, the operation returns 0.

## 4.3 Assumptions and Adversary Model

We follow the common assumptions and adversary model in the literature. Appendix D and E provide details.

# 5 Detailed Universal Channel Rebalancing

This section provides a detailed description of the Universal Channel Rebalancing (UCRb) protocol, which comprises five core operations: *Coin Deposit*, *Coin Allocation*, *Coin Shift*, *Payment*, and *Coin Settlement*, with corresponding algorithms provided in Appendix H.

## 5.1 Coin Deposit

We assume that each user $u_i \in \mathcal{U}$ is connected to the blockchain $\mathcal{B}$, holds a secret and public key pair $(\text{sk}_{u_i}, \text{pk}_{u_i})$, and maintains three distinct lists: the Open Channel list $\mathcal{OC}$, the Coin Shift list $\mathcal{CS}$, and the Close Channel list $\mathcal{CC}$. Additionally, the UCRb smart contract is deployed on the blockchain $\mathcal{B}$, thereby enabling any user to invoke its functions. We now describe the coin deposit routine executed by each user in detail below:

**Initiator $u_i$'s CoinDeposit Routine:** To initiate the coin deposit operation, the user $u_i$ creates a message $\mathsf{m}_{u_i} = \{v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, t_{u_i}\}$, where $v_{u_i}$ is the deposit amount, $f_{u_i}$ is the processing fee, $l_{u_i}$ is the low channel rebalancing fee, $h_{u_i}$ is the high channel rebalancing fee, and $t_{u_i}$ denotes the expiration time. The user then signs the message $\mathsf{m}_{u_i}$ using its secret key $\text{sk}_{u_i}$, resulting in the signature $\sigma_{u_i} \leftarrow \text{Sign}(\text{sk}_{u_i}, \mathsf{m}_{u_i})$. Subsequently, for each neighbor $u_j \in \mathcal{N}(u_i)$, the user generates a channel identifier $c_{\langle u_i,u_j\rangle}$ and initializes the channel balances $\Upsilon_{ij}$ and $\Upsilon_{ji}$, as well as the channel capacity $\complement_{ij}$ (equivalently $\complement_{ji}$), to zero. Afterward, the user $u_i$ sends an open channel request $(\text{open}, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_i}, \sigma_{u_i})$ to each of its neighbors $u_j$. After receiving each neighbor $u_j$'s consent in the form $(\top, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_j}, \sigma_{u_j})$, where $\mathsf{m}_{u_j} = \{v_{u_j}, f_{u_j}, l_{u_j}, h_{u_j}, t_{u_j}\}$ and $\sigma_{u_j}$ is the neighbor's signed consent, the user $u_i$ records the tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \mathsf{m}_{u_i}, \mathsf{m}_{u_j}, \sigma_{u_i}, \sigma_{u_j})$ into its open channel list $\mathcal{OC}$. Subsequently, the user aggregates all signatures, including its own and those of its neighbors, over the message $\mathsf{m}_{u_i}$ to create a multisignature $\sigma \leftarrow \text{SigAgg}(\mathsf{m}_{u_i}, \sigma_{u_i})$. Finally, it invokes the `CoinDeposit` function of the UCRb smart contract by submitting $\text{CoinDeposit}(u_i, \mathcal{N}(u_i), \mathsf{m}_{u_i}, \sigma)$, thereby recording the coin deposit on the blockchain $\mathcal{B}$. At the end of this operation, each user performs only a single on-chain transaction. In contrast to existing approaches, where a separate on-chain operation is required for each individual channel, the proposed method significantly reduces both on-chain costs and transaction latency.

**Neighbor $u_j$'s CoinDeposit Routine:** When user $u_j$ receives the open channel request $(\text{open}, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_i}, \sigma_{u_i})$ from $u_i$, it parses the message $\mathsf{m}_{u_i}$, and verifies two conditions: (i) the on-chain balance of user $u_i$, denoted $\mathcal{B}[u_i] \cdot \text{bal}$, is greater than or equal to the intended deposit amount $v_{u_i}$, i.e., $\mathcal{B}[u_i] \cdot \text{bal} \geq v_{u_i}$; and (ii) the locking time $t_{u_i}$ is strictly greater than the current blockchain time $\mathcal{B}[t_{\text{now}}]$, i.e., $t_{u_i} > \mathcal{B}[t_{\text{now}}]$. Upon satisfying these conditions, if $u_j$ is willing to participate in the off-chain network, it prepares its own message $\mathsf{m}_{u_j} = \{v_{u_j}, f_{u_j}, l_{u_j}, h_{u_j}, t_{u_j}\}$, and initializes the channel

balances $\Upsilon_{ij}$, $\Upsilon_{ji}$, and the channel capacity $\complement_{ji}$ (equivalently $\complement_{ij}$) to zero. Next, $u_j$ signs the message $\mathsf{m}_{u_i}$ received from $u_i$ to provide its consent, i.e., $\sigma_{u_j} \leftarrow \mathsf{Sign}(\mathsf{sk}_{u_j}, \mathsf{m}_{u_i})$. Then, it records the tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \mathsf{m}_{u_i}, \mathsf{m}_{u_j}, \sigma_{u_i}, \sigma_{u_j})$ into its open channel list $OC$, and sends a consent $(\top, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_j}, \sigma_{u_j})$ to $u_i$.

## 5.2 Coin Allocation

After receiving confirmation from the blockchain regarding the coin deposit operation, each user initiates the coin allocation operation. We now describe the coin allocation routine executed by each user in detail below:

**Initiator $u_i$'s CoinAllocation Routine:** The user $u_i$ may allocate coins to each neighbor $u_j \in \mathcal{N}(u_i)$ in arbitrary proportions, provided that the total allocated amount equals the deposited value, i.e., $v_{u_i} = \sum_{j=1}^{|\mathcal{N}(u_i)|} \Upsilon_{ij}$. For the sake of simplicity and generality, we adopt an uniform distribution strategy in this work, where each neighbor $u_j$ receives a share of $\frac{v_{u_i}}{|\mathcal{N}(u_i)|}$. It is flexible and can be extended to support customized allocation strategies based on specific application requirements or network conditions. The design of an optimal coin allocation policy is beyond the scope of this work and is left for future research. To ensure transparency and correctness of the allocation, the user $u_i$ generates a non-interactive zero-knowledge (NIZK) proof, called the coin allocation proof, denoted by $\pi^{ca}_{u_i}$, which can be independently verified by its neighbors. The construction and verification details are provided in Appendix F.1. The user $u_i$ sends the allocated channel balance information, the coin allocation proof, and its signature to each corresponding neighbor $u_j$, in the form of the tuple $(c_{\langle u_i,u_j\rangle}, \pi^{ca}_{u_i}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i})$. Upon receiving a consent from each neighbor $u_j$ in the form $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$, the user $u_i$ records the updated tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \pi^{ca}_{u_i}, \sigma_{u_i}, \sigma_{u_j})$ in its open channel list $OC$. Next, the user $u_i$ aggregates all signatures, including its own and those of its neighbors, to generate a multisignature $\sigma \leftarrow \mathsf{SigAgg}(\pi^{ca}_{u_i}, \sigma_{u_i})$. It then records the tuple $(u_i, \mathcal{N}(u_i), \Upsilon, \pi^{ca}_{u_i}, \sigma)$ in open channel list $OC$, where $\Upsilon$ denotes the final coin allocation set.

**Neighbor $u_j$'s CoinAllocation Routine:** When user $u_j$ receives the message $(c_{\langle u_i,u_j\rangle}, \pi^{ca}_{u_i}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i})$ from user $u_i$, it verifies the validity of the coin allocation proof by checking whether $\mathsf{V}_{\mathsf{NIZK}}(\mathsf{stmt}, \pi^{ca}_{u_i}) \stackrel{?}{=} 1$, where stmt is the public statement associated with the coin allocation proof (refer to Appendix F.1 for more details). Upon successful verification, user $u_j$ records the tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \pi^{ca}_{u_i}, \sigma_{u_i}, \sigma_{u_j})$ in its open channel list $OC$. Subsequently, user $u_j$ signs the message to indicate its consent and sends a consent to $u_i$ in the form of $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$.

## 5.3 Payment

Consider a payment path $\mathcal{P} = \{u_0, \ldots, u_i, \ldots, u_n\}$, where $u_0$ is the sender, $\{u_i\}_{i\in[1,n-1]}$ are intermediary users, and $u_n$ is the receiver. For illustrative purposes, we focus on a concrete scenario where $u_0$ acts as the sender, $\{u_1, u_2, u_3\}$ serve as intermediaries, and $u_4$ is the receiver, as depicted in Figure 1. The generalized protocol applicable to an arbitrary-length path is formally defined in Appendix I. We now present the payment procedure by detailing the execution routines associated with each participating entity in the network.

**Sender $u_0$'s Payment Routine:** To initiate the payment process, the sender $u_0$ performs a setup phase to establish shared secret parameters with the receiver $u_n$. It first generates public parameters: two large prime numbers $p$ and $q$, such that $q \mid (p-1)$, and selects a generator $g$ of the order-$q$ subgroup of the multiplicative group $\mathbb{Z}^*_p$. Then, it samples two secrets $\{s, r\} \stackrel{\$}{\leftarrow} \mathbb{Z}_q$, computes the public commitment $\kappa \leftarrow g^s \pmod p$, and sends the tuple $(\kappa, r, v, \delta)$ to the receiver $u_n$ over a secure channel. Here, $v$ denotes the payment amount, and $\delta$ is an optional incentive fee intended to support channel rebalancing and ensure successful payment execution. When $\delta = 0$ that means the intermediaries need to distribute the rebalancing fee among themselves. Further details on the incentive mechanism are discussed in Section 5.6.

Following the setup phase, the sender $u_0$ initiates the payment by computing the locking parameter, such as the total transfer amount as $v_{01} \leftarrow v + \sum_{i=1}^{3} f_{u_i} + \delta$, where $f_{u_i}$ denotes the processing fee charged by each intermediary $u_i$, and $\delta$ is an optional incentive to support channel rebalancing. The sender then verifies whether the current channel balance $\Upsilon_{01}$ is sufficient to cover the payment, i.e., $v_{01} \leq \Upsilon_{01}$. If this condition is satisfied, $u_0$ proceeds to update the channel balances $\Upsilon_{01} \leftarrow \Upsilon_{01} - v_{01}$ and $\Upsilon_{10} \leftarrow \Upsilon_{10} + v_{01}$. It then samples a random secret $\alpha_{01} \stackrel{\$}{\leftarrow} \mathbb{Z}_q$, and computes the path-specific locking parameters: $\beta_{01} \leftarrow g^{\alpha_{01}} \cdot \kappa^r \pmod p$ and $\gamma_{01} \leftarrow g^{\alpha_{01}} \pmod p$. Subsequently, the sender establishes a contract $\mathsf{PCTLC}(u_0, u_1, v_{01}, t_{01}, \beta_{01}, \gamma_{01}, \pi^{cs}_{01}, \sigma_{01})$ with its immediate neighbor $u_1$, where $t_{01}$ denotes the time lock expiration, $\pi^{cs}_{01}$ is the initial coin shift set (initialized as empty), and $\sigma_{01}$ is the sender's digital signature on the contract parameters. In contrast to the hashed time-lock contracts (HTLCs) used in the Lightning Network [25], our design introduces a Pedersen Commitment-based Time Lock Contract (PCTLC), leveraging Pedersen commitment schemes [24] for constructing the locking mechanism. This approach enhances security by mitigating vulnerabilities such as wormhole attacks [15].

If the immediate neighbor $u_1$ reveals the unlocking parameters $(\varphi, \vartheta_{10}, \pi^{cs}, v_{10}, \sigma_{10})$ before the lock expiration time $t_{01}$, the sender $u_0$ performs a sequence of verification steps to validate the claim. (i) it verifies the correctness of the coin shift operations by checking the proof $\pi^{cs}$, which encapsulates all coin shift activities performed during the locking phase. This verification is carried out using the function $\mathsf{V}_{\mathsf{NIZK}}(\mathsf{stmt}, \pi^{cs})$, where stmt is the public statement associated with the coin shift proof. For a detailed explanation of this process, refer to Section G.1. (ii) it verifies the validity of the opening parameters by checking that $\vartheta_{10} \stackrel{?}{=} \varphi \cdot \beta_{10} \pmod p$ and $\kappa \stackrel{?}{=} \vartheta_{10} \cdot g^{-\alpha_{01}} \pmod p$. (iii) it invokes the function $\mathsf{calculateAmount}(\delta, \pi^{cs}, \pi^{cs}_{10})$ to recompute the expected transfer value and confirms that it matches the claimed amount $v_{10}$. If all verification steps succeed, $u_0$ proceeds to update the channel balances as $\Upsilon_{01} \leftarrow \Upsilon_{01} + (v_{01} - v_{10})$ and $\Upsilon_{10} \leftarrow \Upsilon_{10} - (v_{01} - v_{10})$ and records the updated channel state $(c_{\langle u_0,u_1\rangle}, \Upsilon_{01}, \Upsilon_{10}, \complement_{01}, \pi^{cs}, \sigma_{01}, \sigma_{10})$ in the open channel list $OC$. If any of the verification checks fail, $u_0$ discards the received parameters, retains the previous channel state, and aborts the process.

**Intermediary $u_i$'s Payment Routine:** For generality and clarity, we refer to any intermediate node as $u_i$, its immediate predecessor as $u_{i-1}$, and its immediate successor as $u_{i+1}$. Upon receiving

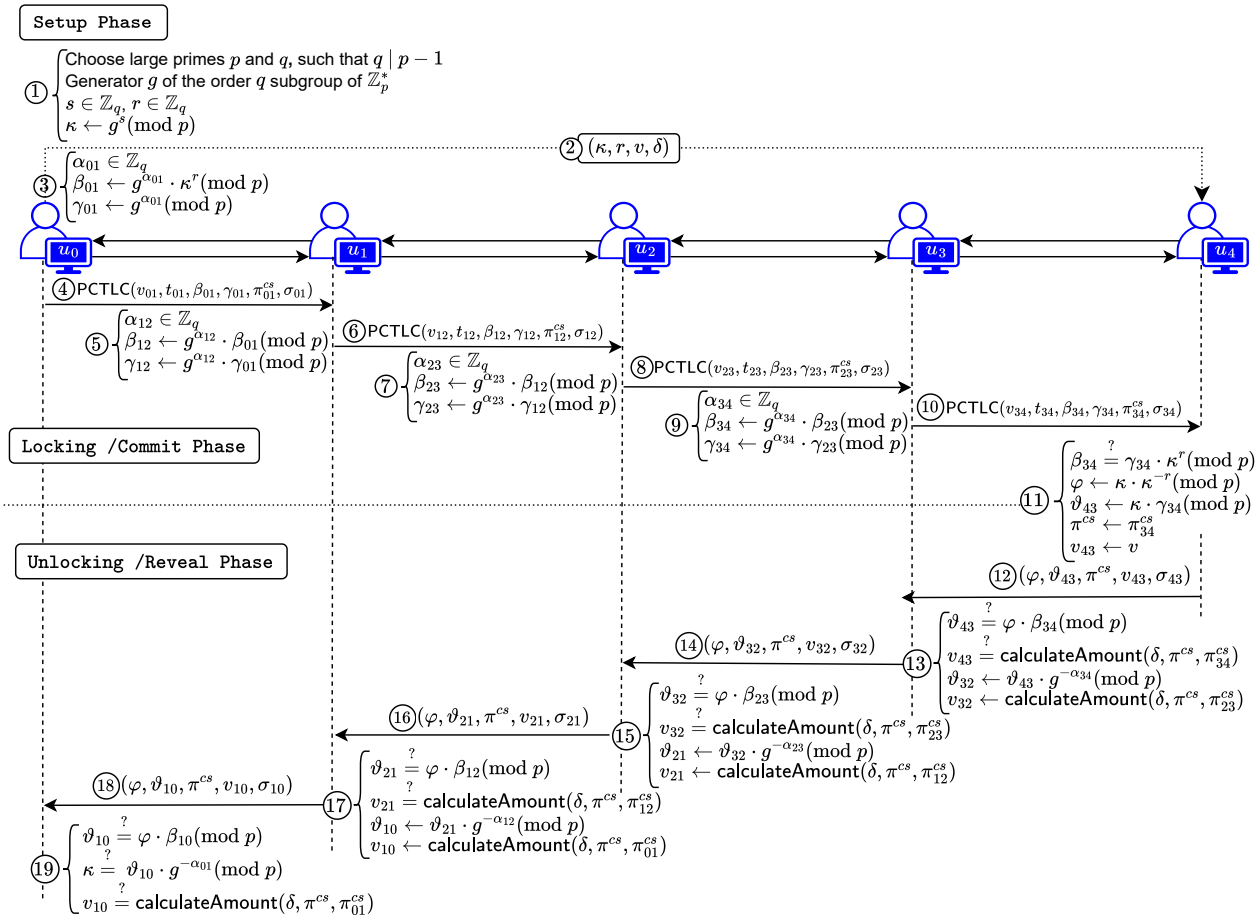



**Figure 1: Universal Channel Rebalancing (UCRb) Payment Protocol**

the contract PCTLC$(u_{i-1}, u_i, v_{\langle i-1,i\rangle}, t_{\langle i-1,i\rangle}, \beta_{\langle i-1,i\rangle}, \gamma_{\langle i-1,i\rangle}, \pi_{\langle i-1,i\rangle}^{cs}, \sigma_{\langle i-1,i\rangle})$ from its predecessor $u_{i-1}$, the intermediate user $u_i$ begins processing the payment. (i) $u_i$ deducts its processing fee $f_{u_i}$ from the received amount, computing the amount to forward as $v_{\langle i,i+1\rangle} \leftarrow v_{\langle i-1,i\rangle} - f_{u_i}$. It then verifies the following conditions: (i) that the available balance on the outgoing payment channel is sufficient, i.e., $v_{\langle i,i+1\rangle} \leq \Upsilon_{\langle i,i+1\rangle}$; (ii) that the lock expiration time is valid and has not expired. If both conditions are satisfied, $u_i$ updates the channel state by setting the balances as: $\Upsilon_{\langle i,i+1\rangle} \leftarrow \Upsilon_{\langle i,i+1\rangle} - v_{\langle i,i+1\rangle}$ and $\Upsilon_{\langle i+1,i\rangle} \leftarrow \Upsilon_{\langle i+1,i\rangle} + v_{\langle i,i+1\rangle}$. It also updates the lock expiration time as $t_{\langle i,i+1\rangle} \leftarrow t_{\langle i-1,i\rangle} - \Delta$, where $\Delta$ represents the maximum delay. Next, $u_i$ generates a fresh secret $\alpha_{\langle i,i+1\rangle} \stackrel{\$}{\leftarrow} \mathbb{Z}_q$ and computes the path-specific locking parameters: $\beta_{\langle i,i+1\rangle} \leftarrow g^{\alpha_{\langle i,i+1\rangle}} \cdot \beta_{\langle i-1,i\rangle} \pmod p$ and $\gamma_{\langle i,i+1\rangle} \leftarrow g^{\alpha_{\langle i,i+1\rangle}} \cdot \gamma_{\langle i-1,i\rangle} \pmod p$. If the verification checks fail due to insufficient balance on the outgoing payment channel, the intermediate node $u_i$ promptly initiates coin shift operation. This operation rebalances the depleted channel by reallocating funds from one or more of its neighboring channels in off-chain, thereby enabling $u_i$ to continue processing the payment without interrupting the transaction flow. The detailed procedure of the coin shift mechanism is provided in Section 5.4. Subsequently, the user $u_i$ establishes the contract PCTLC$(u_i, u_{i+1}, v_{\langle i,i+1\rangle}, t_{\langle i,i+1\rangle}, \beta_{\langle i,i+1\rangle}, \gamma_{\langle i,i+1\rangle}, \pi_{\langle i,i+1\rangle}^{cs}, \sigma_{\langle i,i+1\rangle})$ with its immediate successor $u_{i+1}$. The coin shift proof set $\pi_{\langle i,i+1\rangle}^{cs}$ aggregates any rebalancing operations executed by $u_i$ along with the set $\pi_{\langle i-1,i\rangle}^{cs}$ received from its immediate predecessor $u_{i-1}$. If no coin shift operation is performed by $u_i$, the received coin shift proof set is forwarded without modification.

When the immediate successor $u_{i+1}$ discloses the unlocking parameters $(\varphi, \vartheta_{\langle i+1,i\rangle}, \pi^{cs}, v_{\langle i+1,i\rangle}, \sigma_{\langle i+1,i\rangle})$ before the lock expiration time $t_{\langle i,i+1\rangle}$, the intermediate user $u_i$ undertakes a some of verification steps to validate the authenticity and correctness of the claim. (i) it verifies the integrity of the coin shift operations by validating the coin shift proof set $\pi^{cs}$, which contains all coin shift operation performed during the locking phase. This is done using the verification function $\mathsf{V}_{\mathsf{NIZK}}(\mathsf{stmt}, \pi^{cs})$, where stmt represents the publicly known statement corresponding to the coin shift proof (for more details is provided in Appendix G.1). (ii) it checks the correctness of the opening parameters by verifying the conditions $\vartheta_{\langle i+1,i\rangle} \stackrel{?}{=} \varphi \cdot \beta_{\langle i+1,i\rangle} \pmod p$, which ensure the consistency of the cryptographic commitments and the path-specific locking structure.

(iii) the function calculateAmount$(\delta, \pi^{cs}, \pi^{cs}_{\langle i+1,i\rangle})$ is invoked to recompute the expected transfer value, and the result is cross-checked against the claimed amount $v_{\langle i+1,i\rangle}$. If all verification steps succeed, $u_i$ updates the channel balances as follows: $\Upsilon_{\langle i,i+1\rangle} \leftarrow \Upsilon_{\langle i,i+1\rangle} + \Big(v_{\langle i,i+1\rangle} - v_{\langle i+1,i\rangle}\Big)$, and $\Upsilon_{\langle i+1,i\rangle} \leftarrow \Upsilon_{\langle i+1,i\rangle} - \Big(v_{\langle i,i+1\rangle} - v_{\langle i+1,i\rangle}\Big)$. The updated channel state $(c_{\langle u_i,u_{i+1}\rangle}, \Upsilon_{\langle i,i+1\rangle}, \Upsilon_{\langle i+1,i\rangle}, \complement_{\langle i,i+1\rangle}, \pi^{cs}, \sigma_{\langle i,i+1\rangle}, \sigma_{\langle i+1,i\rangle})$ is then stored in the open channel list $OC$. Subsequently, user $u_i$ computes the unlocking parameters required to settle the transaction with its immediate predecessor $u_{i-1}$. Specifically, it derives the value $\vartheta_{\langle i,i-1\rangle} \leftarrow \vartheta_{\langle i+1,i\rangle} \cdot g^{\alpha_{\langle i,i+1\rangle}} \pmod p$, and invokes the function calculateAmount$(\delta, \pi^{cs}, \pi^{cs}_{\langle i-1,i\rangle})$ to compute the amount to be claimed from the predecessor. Thereafter, $u_i$ updates the state of the channel shared with $u_{i-1}$ is as follows: $\Upsilon_{\langle i-1,i\rangle} \leftarrow \Upsilon_{\langle i-1,i\rangle} - \Big(v_{\langle i-1,i\rangle} - v_{\langle i,i-1\rangle}\Big)$, and $\Upsilon_{\langle i,i-1\rangle} \leftarrow \Upsilon_{\langle i,i-1\rangle} + \Big(v_{\langle i-1,i\rangle} - v_{\langle i,i-1\rangle}\Big)$. The updated channel state $(c_{\langle u_{i-1},u_i\rangle}, \Upsilon_{\langle i-1,i\rangle}, \Upsilon_{\langle i,i-1\rangle}, \complement_{\langle i-1,i\rangle}, \pi^{cs}, \sigma_{\langle i-1,i\rangle}, \sigma_{\langle i,i-1\rangle})$, is then recorded in the open channel list $OC$. After the channel update, $u_i$ releases the locked amount $v_{\langle i,i+1\rangle}$ to its successor $u_{i+1}$ and concurrently reveals the unlocking tuple $(\varphi, \vartheta_{\langle i,i-1\rangle}, \pi^{cs}, v_{\langle i,i-1\rangle}, \sigma_{\langle i,i-1\rangle})$ to $u_{i-1}$, which enables $u_{i-1}$ to validate the transaction and release the payment. This process allows $u_i$ to receive the corresponding payment $v_{\langle i,i-1\rangle}$ from its predecessor. If any of the above steps fail at any stage, discards the received unlocking request, reverts to the previously stored channel state, and $u_i$ aborts the process.

**Receiver $u_n$'s Payment Routine:** Upon receiving the contract PCTLC$(u_{n-1}, u_n, v_{\langle n-1,n\rangle}, t_{\langle n-1,n\rangle}, \beta_{\langle n-1,n\rangle}, \gamma_{\langle n-1,n\rangle}, \pi^{cs}_{\langle n-1,n\rangle}, \sigma_{\langle n-1,n\rangle})$ from its immediate predecessor $u_{n-1}$, the receiver $u_n$ performs a sequence of verifications to ensure the validity and consistency of the received contract. (i) It verifies that the received amount matches the original payment value $v$ as specified by the sender $u_0$ during the setup phase, i.e., $v \stackrel{?}{=} v_{\langle n-1,n\rangle}$. (ii) It checks whether the time-lock associated with the contract is still valid and has not expired. (iii) It validates the correctness of the coin shift operations executed by intermediate users in this phase. This is achieved by verifying the coin shift proof set $\mathsf{V}_{\mathsf{NIZK}}(\text{stmt}, \pi^{cs}_{\langle n-1,n\rangle})$, where stmt is the publicly known statement corresponding to the coin shift operations (for more details is provided in Appendix G.1). (iv) It checks the correctness of the opening parameters by verifying the consistency of the commitment, i.e., $\beta_{\langle n-1,n\rangle} \stackrel{?}{=} \gamma_{\langle n-1,n\rangle} \cdot \kappa^r \pmod p$. This ensures that the revealed decommitment values align with the original commitments and the path-specific locking structure. If all the above checks succeed, $u_n$ accepts the payment and finalizes the transaction. It updates the corresponding payment channel balances are $\Upsilon_{\langle n-1,n\rangle} \leftarrow \Upsilon_{\langle n-1,n\rangle} - v_{\langle n-1,n\rangle}$, and $\Upsilon_{\langle n,n-1\rangle} \leftarrow \Upsilon_{\langle n,n-1\rangle} + v_{\langle n-1,n\rangle}$, and updates the channel capacity is $\complement_{\langle n-1,n\rangle} \leftarrow \Upsilon_{\langle n-1,n\rangle} + \Upsilon_{\langle n,n-1\rangle}$. Subsequently, the receiver computes the path-specific unlocking parameters $\varphi \leftarrow \kappa \cdot \kappa^{-r} \pmod p$, and $\vartheta_{\langle n,n-1\rangle} \leftarrow \kappa \cdot \gamma_{\langle n-1,n\rangle} \pmod p$, and sets the claiming amount and proof set as $v_{\langle n,n-1\rangle} \leftarrow v$, and $\pi^{cs} \leftarrow \pi^{cs}_{\langle n-1,n\rangle}$. The receiver $u_n$ then records the updated state by storing the tuple $(c_{\langle n-1,n\rangle}, \Upsilon_{\langle n-1,n\rangle}, \Upsilon_{\langle n,n-1\rangle}, \complement_{\langle n-1,n\rangle}, \pi^{cs}, \sigma_{\langle n-1,n\rangle}, \sigma_{\langle n,n-1\rangle})$ in its local open channel list $OC$. Finally, it discloses the path-specific unlocking parameters $(\varphi, \vartheta_{\langle n,n-1\rangle}, v_{\langle n,n-1\rangle}, \sigma_{\langle n,n-1\rangle})$ to its immediate predecessor $u_{n-1}$ to claim for releasing the coins.

## 5.4 Coin Shift

The coin shift operation is initiated by any intermediate user $u_i$ (the initiator) along the payment path. This is occur when it cannot forward a payment to its immediate successor $u_j$ (the target neighbor) due to insufficient funds in the outgoing channel $c_{\langle u_i,u_j\rangle}$, despite having adequate liquidity across other channels. To resolve this localized liquidity bottleneck, the initiator dynamically rebalances its channels by shifting coins from one or more source neighbors $u_k \in \mathcal{N}(u_i) \setminus \{u_j\}$ into $c_{\langle u_i,u_j\rangle}$. This off-chain rebalancing ensures uninterrupted payment flow without requiring on-chain settlement, offering an efficient, verifiable, and privacy-preserving solution, unlike prior approaches such as Shaduf [12] and Horcrux [27]. This work introduces an on-demand rebalancing mechanism that dynamically adjusts channel balances during payment execution when liquidity is insufficient. The approach can be extended to a proactive, payment-independent rebalancing system that maintains channel liquidity continuously. We defer this generalized mechanism to future work. We now present the coin shift procedure by outlining the execution routines of each participating entity involved in the operation, as illustrated in Figure 2. The complete step-by-step procedure is formally described in Appendix J.

**Initiator $u_i$'s CoinShift Routine:** As illustrated in Figure 2, the intermediate user $u_i$ encounters insufficient balance in the outgoing channel $c_{\langle u_i,u_j\rangle}$ to forward a payment, it initiates a coin shift operation. The initiator $u_i$ selects one or more source neighbors (e.g., external neighbor $u_k$) based on available liquidity, prioritizing those with higher balances. While the current construction considers both in-path and external neighbors, optimal neighbor selection strategies remain for future investigation. To ensure secure execution, $u_i$ establishes similar locking mechanism to the payment operation (excluding PCTLC contracts). The initiator $u_i$ samples a random secret $\alpha'_{ik} \stackrel{\$}{\leftarrow} \mathbb{Z}_q$, computes the commitments $\beta'_{ik} \leftarrow g^{\alpha'_{ik}} \cdot \beta_{hi} \pmod p$ and $\gamma'_{ik} \leftarrow g^{\alpha'_{ik}} \cdot \gamma_{hi} \pmod p$. The initiator $u_i$ then sends authenticated coin shift requests to both neighbors: to the source $u_k$ it sends CoinShift$(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \beta'_{ik}, \gamma'_{ik}, \sigma^i_{ik}, \sigma^i_{ij})$, and to the target $u_j$ it sends CoinShift$(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ik}, \sigma^i_{ij})$. Here, $v^i_{kj}$ denotes the amount being shifted from $c_{\langle u_i,u_k\rangle}$ to $c_{\langle u_i,u_j\rangle}$, while $\sigma^i_{ik}$ and $\sigma^i_{ij}$ represent $u_i$'s signatures for channels $c_{\langle u_i,u_k\rangle}$ and $c_{\langle u_i,u_j\rangle}$, respectively. We use primed notation $(\beta'_{ik}, \gamma'_{ik})$ for commitments and distinct signature variables to clearly differentiate these coin-shift-specific parameters from those used in regular other operations.

Upon receiving consent from the source neighbor $u_k$ in form of $(\top, c_{\langle u_k,u_i\rangle}, \beta'_{ki}, \gamma'_{ki}, \sigma^k_{ik}, \sigma^j_{ij})$, and from the target neighbor $u_j$ as $(\top, c_{\langle u_i,u_j\rangle}, \sigma^j_{ij}, \sigma^k_{ik})$, the initiator $u_i$ proceeds to verify the correctness and authenticity of the received response. These verification steps confirm that: both neighbors have mutually acknowledged the coin shift proposal, have consented to participate, and the attached signatures are valid and mutually consistent. Following successful verification, initiator $u_i$ updates channel balances by computing: $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - v^i_{kj}$, $\complement'_{ik} \leftarrow \complement_{ik} + v^i_{kj}$, $\Upsilon'_{ij} \leftarrow \Upsilon_{ij} + v^i_{kj}$, $\complement'_{ij} \leftarrow \complement_{ij} - v^i_{kj}$ and records updated states $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon_{ki}, \complement'_{ik}, \sigma^i_{ik}, \sigma^k_{ki})$

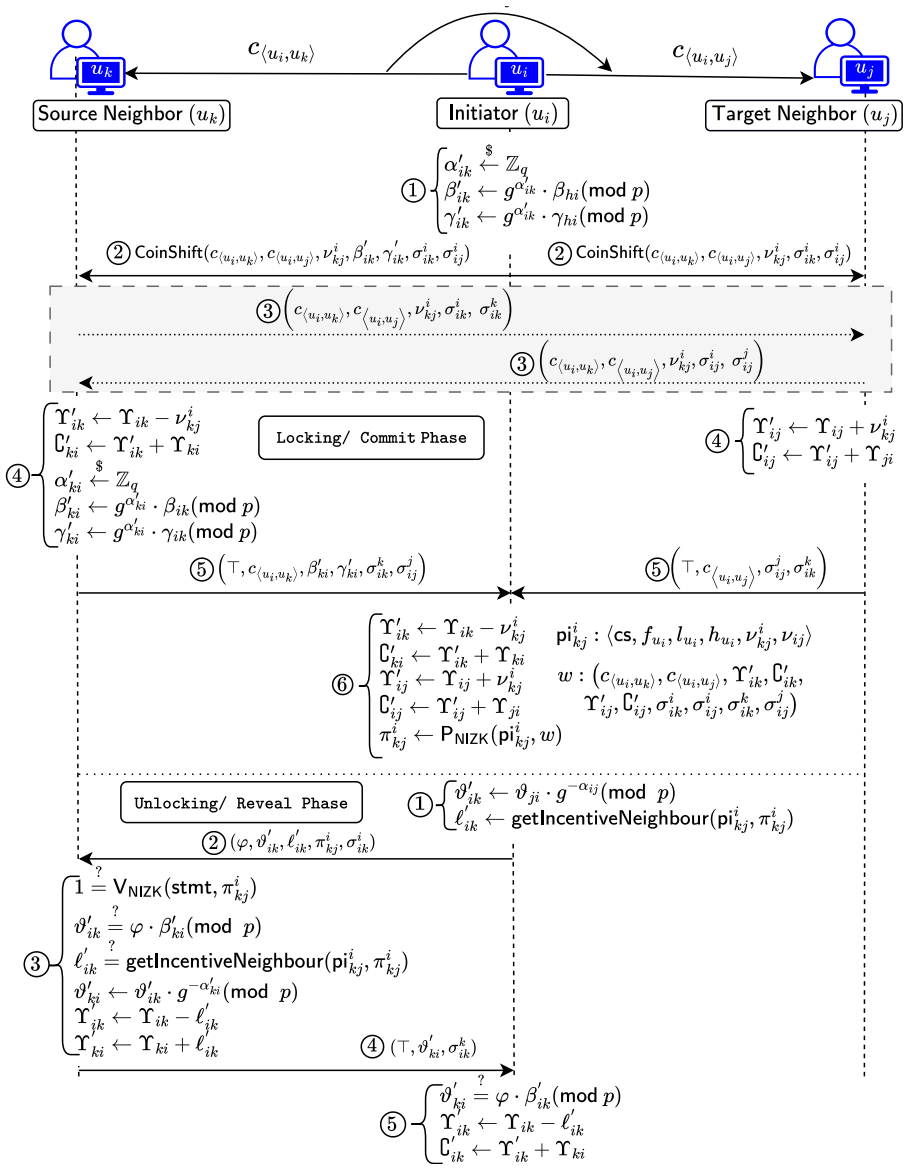



**Figure 2: Channel Rebalancing using Coinshift Operation**

and $(c_{\langle u_i,u_j\rangle}, \Upsilon'_{ij}, \Upsilon_{ji}, \complement'_{ij}, \sigma^i_{ij}, \sigma^j_{ij})$ in its local open channel set $OC$. To ensure verifiable while preserving privacy, the initiator $u_i$ generates a coin shift proof $\pi^i_{kj}$ that: (i) cryptographically attests to honest operation execution; (ii) preserves source neighbor $u_k$'s anonymity from non-participating users; and (iii) provides necessary authentication for payment completion (formal construction in Appendix G.1). The coin shift operation transfers $\nu^i_{kj}$ coins from the channel $c_{\langle u_i,u_k\rangle}$ to $c_{\langle u_i,u_j\rangle}$, while enforcing a security hold: the source neighbor $u_k$ withholds unlocking parameters of its commitment until it receives designated incentive payment $\ell'_{ik}$ from initiator $u_i$.

Upon receiving the PCTLC contract's unlocking parameters from target neighbor $u_j$, initiator $u_i$ verifies all conditions as specified in Section 5.3. The initiator then computes the unlocking parameters for source neighbor $u_k$, including the decommitment value $\vartheta'_{ik} \leftarrow \vartheta_{ji} \cdot g^{-\alpha_{ij}} (\pmod p)$ and the incentive amount $\ell'_{ik} \leftarrow \mathsf{getIncentive}(\pi^i_{kj}, \mathsf{pi}^i_{kj})$. Subsequently, $u_i$ reveals the unlocking tuple $(\varphi, \vartheta'_{ik}, \ell'_{ik}, \pi^{cs}, \sigma^i_{ik})$ to $u_k$ to enable completion of the coin shift process. When $u_k$ receives this tuple, it verifies the condition $\vartheta'_{ki} \stackrel{?}{=} \varphi \cdot \beta'_{ik} (\mathrm{mod} p)$. If satisfied, it updates the channel states as $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - \ell'_{ik}$ and $\complement'_{ik} \leftarrow \Upsilon'_{ik} + \Upsilon_{ki}$, then proceeds to the PCTLC contract reveal phase (as detailed in Section 5.3). Otherwise, if the verification fails, the protocol restores the previous channel

states and aborts the operation. The procedure for handling multiple coin shift operations initiated by a single initiator is detailed in Appendix 3. This ensures seamless payment continuation while preserving balance security guarantees and maintaining the integrity and consistency of the off-chain network state.

**Source Neighbor $u_k$'s CoinShift Routine:** Upon receiving the coin shift request $\mathsf{CoinShift}(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \beta'_{ik}, \gamma'_{ik}, \sigma^i_{ik}, \sigma^i_{ij})$ from the initiator $u_i$, the source neighbor $u_k$ verifies that the requested amount $v^i_{kj}$ does not exceed the available liquidity in the channel $c_{\langle u_i,u_k\rangle}$, i.e., $\Upsilon_{ik} \geq v^i_{kj}$. If this condition holds and $u_k$ agrees to the request, it initiates a bilateral verification exchange. Specifically, $u_k$ sends an authorization tuple $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ik}, \sigma^k_{ik})$ to the target neighbor $u_j$ over a dedicated communication channel (as illustrated in the shaded region of Figure 2). This bidirectional exchange of verification data confirms the legitimacy of the coin shift request, establishes mutual agreement among participants, and mitigates the risk of misbehavior by the initiator. Upon receiving the response tuple $(c_{\langle u_i,u_j\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ij}, \sigma^j_{ij})$ from $u_j$, the source neighbor $u_k$ performs the necessary verification on the received data. Once verified, it updates its local channel balance as $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - v^i_{kj}$ and channel capacity as $\complement'_{ik} \leftarrow \Upsilon'_{ik} + \Upsilon_{ki}$. To proceed with the atomic coin shift, $u_k$ samples a fresh random secret $\alpha'_{ki} \xleftarrow{\$} \mathbb{Z}_q$ and computes the corresponding commitments: $\beta'_{ki} \leftarrow g^{\alpha'_{ki}} \cdot \beta'_{ik} (\mathrm{mod} p)$, and $\gamma'_{ki} \leftarrow g^{\alpha'_{ki}} \cdot \gamma'_{ik} (\mathrm{mod} p)$. Following these computations, $u_k$ records the tuple $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon_{ki}, \complement'_{ik}, \sigma^i_{ik}, \sigma^k_{ik})$ in its local open channel list $OC$, and stores the tuple $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ik}, \sigma^k_{ik}, \sigma^i_{ij}, \sigma^j_{ij})$ in the local coin shift list $\mathcal{CS}$. Then, $u_k$ sends the locking parameters as a consent tuple $(\top, c_{\langle u_i,u_k\rangle}, \beta'_{ki}, \gamma'_{ki}, \sigma^k_{ik}, \sigma^j_{ij})$ to the initiator $u_i$.

Upon receiving the unlocking parameters $(\varphi, \vartheta'_{ik}, \ell'_{ik}, \pi^i_{kj}, \sigma^i_{ik})$ from the initiator $u_i$, the protocol performs a series of verification and update steps to ensure the integrity of the coin shift operation. (i) It validates the coin shift proof, verifier executes the verification function $\mathsf{V}_{\mathsf{NIZK}}(\mathsf{stmt}, \pi^i_{kj})$ to confirm the proof's correctness (details provided in Appendix G.1). (ii) It verifies that the unlocking key $\vartheta'_{ik}$ is correctly derived by checking if the equality $\vartheta'_{ik} = \varphi \cdot \beta'_{ki} \bmod p$ holds. (iii) It confirms the validity of the claimed incentive amount $\ell'_{ik}$ by comparing it against the expected value computed using the function $\mathsf{getIncentiveNeighbor}(\pi^i_{kj}, \pi^i_{kj})$. If all these condition holds, it computes the unlocking parameter as $\vartheta'_{ki} \leftarrow \vartheta'_{ik} \cdot g^{-\alpha}$ and proceeds to update the channel balances: $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - \ell'_{ik}$ and $\Upsilon'_{ki} \leftarrow \Upsilon_{ki} + \ell_{ik}$. Subsequently, with the user's consent, it discloses the opening parameter to the initiator $u_i$ in the form $(\top, \vartheta'_{ki}, \sigma^k_{ik})$, thereby completing the coin shift operation.

**Target Neighbor $u_j$'s CoinShift Routine:** Upon receiving the coin shift request $\mathsf{CoinShift}(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \beta'_{ik}, \gamma'_{ik}, \sigma^i_{ik}, \sigma^i_{ij})$ from the initiator $u_i$, the target neighbor $u_j$ verifies that the requested coin shift amount does not exceed the on-chain deposited liquidity, i.e., $\mathcal{B}[u_i] \geq v^i_{kj}$. If this condition holds and $u_j$ agrees to proceed, it initiates a bilateral verification exchange with the source neighbor $u_k$ by sending an authorization tuple $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ij}, \sigma^j_{ij})$ over a dedicated communication channel (as depicted in the shaded region of Figure 2). This bidirectional exchange establishes mutual agreement among the involved parties and mitigates the risk of misbehavior by the initiator. Upon receiving the corresponding response tuple $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ik}, \sigma^k_{ik})$ from $u_k$, the target neighbor $u_j$ verifies the received data. Once validated, $u_j$ updates its local channel balance as $\Upsilon'_{ij} \leftarrow \Upsilon_{ij} - v^i_{kj}$ and recalculates the channel capacity as $\complement'_{ij} \leftarrow \Upsilon'_{ij} + \Upsilon_{ji}$. Subsequently, $u_j$ records the tuple $(c_{\langle u_i,u_j\rangle}, \Upsilon'_{ij}, \Upsilon_{ji}, \complement'_{ij}, \sigma^i_{ij}, \sigma^j_{ij})$ in its local open channel list $OC$, and stores $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v^i_{kj}, \sigma^i_{ij}, \sigma^j_{ij}, \sigma^i_{ik}, \sigma^k_{ik})$ in its local coin shift list $\mathcal{CS}$. Finally, $u_j$ sends tuple $(\top, c_{\langle u_i,u_k\rangle}, \beta'_{ki}, \gamma'_{ki}, \sigma^k_{ik}, \sigma^j_{ij})$ to initiator $u_i$ as consent to completing the coin shift operation.

## 5.5 Coin Settlement

Here, we present the coin settlement routine executed by each user.

**Initiator $u_i$'s CoinSettlement Routine:** When a user $u_i$ wishes to initiate the *coin settlement* operation, either proactively before the locking period expires or automatically upon its expiration, it must first obtain consent from all neighboring users $u_j \in \mathcal{N}(u_i)$. This consent is essential to guarantee the correctness, integrity, and balance security of the off-chain commitment finalization process. To begin the process, user $u_i$ constructs the *final state*, denoted by $\mathsf{F}_{\mathsf{state}}$, which aggregates the latest states of all its open channels with neighbors. For each neighbor $u_j \in \mathcal{N}(u_i)$, the channel state is represented as the tuple $\mathsf{state}(c_{\langle u_i,u_j\rangle}) := (c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i}, \sigma_{u_j})$. The complete final state is then formed by concatenating these channel states: $\mathsf{F}_{\mathsf{state}} \leftarrow \{\mathsf{state}(c_{\langle u_i,u_1\rangle}) \parallel \cdots \parallel \mathsf{state}(c_{\langle u_i,u_n\rangle})\}$, where $\mathcal{N}(u_i) = \{u_1, \ldots, u_n\}$. This aggregated state serves as the basis for neighbor verification and the final on-chain settlement. Subsequently, user $u_i$ digitally signs the aggregated final state $\mathsf{F}_{\mathsf{state}}$ using its secret key: $\sigma_{u_i} \leftarrow \mathsf{Sign}(\mathsf{sk}_{u_i}, \mathsf{F}_{\mathsf{state}})$ and distributes $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \sigma_{u_i})$ to each neighboring user $u_j$ to request their consent.

It then sends the tuple $\left(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \sigma_{u_i}\right)$ to each neighboring user $u_j \in \mathcal{N}(u_i)$ to request their cryptographic consent for the coin settlement. Upon receiving a valid consent $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$ from a neighbor $u_i$ records the corresponding final state of the channel information by inserting the tuple $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \mathsf{state}(c_{\langle u_i,u_j\rangle}), \sigma_{u_i}, \sigma_{u_j})$ into its closing channel list $CC$. Once all required consents are collected from its neighbors, user $u_i$ aggregates all the signatures into a single multisignature: $\sigma \leftarrow \mathsf{SigAgg}(\mathsf{F}_{\mathsf{state}}, \sigma_{u_i})$. This aggregated signature includes the endorsements of all neighboring users along with $u_i$'s own signature. Finally, user $u_i$ invokes the `CoinSettlement` function of the UCRb smart contract by submitting $\mathsf{CoinSettlement}(u_i, \mathsf{F}_{\mathsf{state}}, \sigma)$. This transaction finalizes the coin settlement process and immutably records the outcome on the blockchain $\mathcal{B}$, thereby ensuring accountability, verifiability, and transparency, while facilitating the correct distribution of coins as per the finalized off-chain commitments. Finally, each user executes only a single on-chain transaction, irrespective of the number of channels it maintains. This significantly reduces on-chain gas costs and transaction latency, offering a more scalable and efficient solution for off-chain payment systems.

**Neighbor $u_j$'s CoinSettlement Routine:** Upon receiving the coin settlement request $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\text{state}}, \sigma_{u_i})$ from user $u_i$, the neighboring user $u_j$ verifies the consistency of the received channel state. Specifically, it checks whether the channel state $\text{state}(c_{\langle u_i,u_j\rangle})$ included in the aggregated final state $\mathsf{F}_{\text{state}}$ matches its own locally recorded state in the open channel list $OC$, i.e., $\mathsf{F}_{\text{state}}[\text{state}(c_{\langle u_i,u_j\rangle})] \stackrel{?}{=} OC_{u_j}[\text{state}(c_{\langle u_i,u_j\rangle})]$. If the consistency check is successful, user $u_j$ proceeds to record the settlement data by inserting a tuple $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\text{state}}, \text{state}(c_{\langle u_i,u_j\rangle}), \sigma_{u_i}, \sigma_{u_j})$ into its local closing channel list $CC$. This tuple captures the agreed-upon channel identifier, the complete final state $\mathsf{F}_{\text{state}}$, the corresponding local channel state, and the signatures of both users involved in the settlement. Subsequently, $u_j$ generates a digitally signed consent message $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$, where $\top$ indicates approval, and $\sigma_{u_j}$ is the signature over the final state using $u_j$'s secret key. This message is sent back to user $u_i$, confirming $u_j$'s agreement with the proposed settlement and authorizing the finalization process.

## 5.6 Incentive Distribution Cases

In our proposed incentive mechanism Algorithm 4, we assume that all nodes behave rationally and aim to maximize the rewards they receive. We consider two distinct scenarios: one in which the sender rewards the nodes that participate in coin shifting, and another where intermediate nodes are responsible for rewarding the nodes that have performed coin shifts. The reward allocation is based on two primary factors: the number of coin shift operations performed by a node, and the total amount of coins shifted by that node. During the *lock phase*, each node locks an additional amount $\delta$, which it intends to use to reward downstream nodes during the *unlock phase*. Specifically, in the lock phase, if node $j$ receives $\pi_{ij}^{cs}$ from node $i$, node $j$ will utilize this value to compute the incentive it receives from node $i$. Incentives are distributed proportionally, based on each node's contribution to the overall coin shifting in the transaction — both in terms of the number of coin shifts and the total amount shifted. Each node is configured with a lower bound $l$, an upper bound $h$, and a processing fee $f$, where the node is willing to contribute an incentive between $l$ and $h$ percent of $f$.

**Case 1 (Sender-Funded Model):** In the *unlock phase*, a node $j$ receives the total amount $v + \tau_f + \tau_r$, where $v$ is the actual amount to be transferred, $\tau_f$ denotes the cumulative processing fees of all nodes ahead of $j$ (including $j$ itself), and $\tau_r$ represents the total rewards for all nodes ahead of $j$ (again, including $j$) that have participated in coin shifting. This process continues recursively until it reaches the sender, who is ultimately responsible for covering both the processing fees and the incentive rewards for all participating nodes.

**Case 2 (Relay-Funded Model):** During the *unlock phase*, a node receives the final amount as $v + \tau_f - \tau_l + \tau_r$, where $v$ is the actual amount to be transferred, $\tau_f$ represents the cumulative processing fees of all nodes ahead of node $j$ (including $j$ itself), $\tau_l$ is the total incentive given by node $j$ and the nodes ahead of it to the nodes behind, and $\tau_r$ is the total incentive received from the nodes behind node $j$ to the node ahead of $j$ (including $j$ itself). This process continues recursively until it reaches the sender.

**Table 2: Smart Contract Implementation**

| Function | Neighbors | Operations | Gas Cost |
|---|---|---|---|
| Deployment | $k$ | - | 420K + 780K |
| Deposit | $k$ | $[k] \times \mathcal{V}_\sigma + H$ | 30K + 10K $\times k$ |
| Settlement | $k$ | $[k] \times \mathcal{V}_\sigma + k \times H$ | 30K + 13K $\times k$ |

**Table 3: ZK Proof Performance Analysis**

| Proof Type | Number of Constraints | Prove Time | Verifier Time | Proof Size |
|---|---|---|---|---|
| Coin Allocation | 5 K | < 1 s | < 0.2 s | 814 B |
| Coin Shift | 22 K | 1.2 s | < 0.2 s | 814 B |

# 6 Performance Analysis

## 6.1 Implementation

*6.1.1 Smart Contract Implementation.* The proposed method is compatible with any public blockchain platform, including Bitcoin and Ethereum. For our prototype, we implement the on-chain verifier of the payment protocol as a smart contract compatible with the Ethereum Virtual Machine (EVM). Our implementation uses the Schnorr signature scheme, and for efficient multisignature verification, we adopt the optimized Schnorr verifier introduced in [11]. Table 2 reports the gas costs for the three main functions of the smart contract: UCRbDeployment, CoinDeposit, and CoinSettlement.

*6.1.2 [ZK] Implementation Details.* We implemented the proposed ZK statements (Definition 3 and Definition 4) using `circom` DSL and employed the `snarkjs` library for proof generation and verification. The implementation is open source and available at GitHub[5]. Although our ZKP design is not restricted to a specific zkSNARK scheme, we used the Groth16 in our prototype to achieve a constant-size proof. For efficient multisignature verification, we integrated the library introduced in [22], which provides verifiable Schnorr signature support in `circom`. Table 3 presents the performance evaluation of our prototype implementation.

## 6.2 Evaluation

Existing channel rebalancing methods aim to improve the throughput of PCNs by transferring funds from adjacent channels to depleted ones. This process alleviates bottlenecks by replenishing coins in depleted hops, thereby enabling more successful payments. However, the balance skewness introduced by Multi-Hop Payments (MHPs) can exacerbate the problem of channel depletion, as uneven fund distribution reduces the availability of liquidity where it is most needed.

To address this issue, UCRb introduces a novel mechanism that leverages coin shifts from adjacent channels, including those outside the immediate payment path, while ensuring transaction atomicity. By dynamically shifting coins into channels about to be depleted, UCRB enhances payment success rates and reduces overall channel depletion.

[5]https://github.com/UCRb/Universal-Channel-Rebalancing

To evaluate the performance of UCRb particularly in comparison to existing schemes such as Lightning Network (LN) [25], Revive [14], Shaduf [12], and Horcrux [27]. we design a comprehensive set of experiments with a focus on the following key aspects:

(1) How does UCRb improve the performance of the PCN in terms of success ratio, transaction volume, and mitigation of channel depletion?
(2) How does UCRb enhance the long-term operational efficiency and sustainability of the PCN?
(3) How does UCRb ensure fair incentive distribution for nodes that participate in coin shifting?

**Simulation Setup**: Simulation is conducted on a workstation equipped with an Intel Xeon CPU E5 - 4650 v4 @ 2.20 GHz and 256 GB of RAM. We used the Python3 programming language to implement off-chain operations, and the networkx module is used to handle graph-related operations. The implementation is open source and available at GitHub[5]. Our test network is constructed based on a snapshot of the LN, the largest off-chain payment network, comprising 10,529 nodes and 38,910 channels. Due to the unavailability of initial channel balance data, we assume an equal distribution of funds between the two endpoints of each channel. This uniform balance allocation is consistently applied across all experiments to eliminate potential bias in the results. For transaction data, we utilize a payment value dataset randomly sampled from real-world Bitcoin transaction traces recorded between March 1 and March 31, 2021. This dataset contains over 2.65 million micropayment transactions, providing a realistic and diverse workload for evaluating network performance.

**Ways of Evaluation and Comparison.** Following the methodology and simulation setup described above, we conduct a comprehensive evaluation that includes the following aspects:

(1) We evaluate the performance of UCRb over 20 consecutive batches, with channel capacities varying proportionally in each evaluation. This setup is intended to assess whether UCRb achieves superior performance in terms of deposit reduction compared to other schemes. Under the same conditions, we also evaluate the performance of LN, OPT-Revive, Shaduf (using the high-to-low strategy, referred to as HL-Shaduf), and Horcrux for comparison.
(2) We further test the long-term performance of UCRb over 200 consecutive batches with a fixed channel capacity of 8. For comparison, the performance of LN, OPT-Revive, HL-Shaduf, and Horcrux is also evaluated under the same experimental settings.
(3) We calculated the incentive distribution for the nodes, assuming a uniform processing fee of 1. The lower and upper percentage values for each node are fixed at 5% and 10%, respectively. A sample of incentive data from randomly selected transactions is presented in Table 4.

In our simulation framework, each batch consists of 50,000 payment transactions. For every transaction within a batch, a sender-receiver pair is randomly selected along with the corresponding payment value, and the payment is routed through the shortest feasible path. To simplify the evaluation, we assume uniform transaction fees across all network nodes. Given the stochastic nature of the simulation, each experiment is repeated 10 times, and the results are averaged to ensure reliability. To assess performance, we employ two primary metrics: the success ratio of payments, which reflects improvements in network throughput, and the number of depleted channels, which serves as an indicator of the network's sustainability.

**Evaluation Results.** Using the defined performance metrics, we initialize the network model with uniformly distributed channel balances and conduct a statistical evaluation of four different schemes under various conditions. The outcomes of these evaluations are visually summarized in Figures [3 - 6] , allowing for a clear comparison of the schemes' effectiveness in terms of throughput and sustainability.

**Comparison of Success Ratio.** We vary the channel capacity from 1x to 25x and present the success ratios of five different schemes in Figure 3. when Compared to LN, Revive and Shaduf achieve success ratio improvements in the range of 4%–8% and 13%–20%, respectively, across various channel capacities. Horcrux offers a improvement of 21%–47%. Notably, UCRb further enhances performance by 20%–45% over Horcrux. Furthermore, under lower channel capacity conditions, UCRb achieves up to 5.7x, 5.0x, 2.2x, and 1.4x improvement in success ratio compared to LN, Revive, Shaduf, and Horcrux, respectively.

**Comparison of depletion mitigation.** Figure 4. illustrates the number of depleted channels for five different schemes under varying channel capacities, ranging from 1x to 25x. Both Revive and Shaduf result in more than a thousand depleted channels, while UCRb maintains an almost negligible count. Specifically, Revive accounts for 6%–17% and Shaduf for 4%–13% of channel depletions in the PCN, depending on the capacity level. The success of various MHPs facilitated by Revive and Shaduf leads to imbalances in many channels. The curves of Horcrux and LN overlap with that of UCRb in the graph.

**Comparison of Deposit.** Figure 3. illustrates the channel capacity required by each of the five schemes to achieve equivalent performance. For instance, to reach a success ratio of 85%, Shaduf requires a channel capacity factor of 25, while Horcrux and UCRb require only 7 and 2, respectively. This implies that Horcrux and UCRb can match Shaduf's performance using just 0.28x and 0.08x of the capacity, leading to a reduction in overall deposit requirements by approximately 72% and 92%, respectively.

**Comparison Under Long-Term Operation.** The results of the long-term evaluation are presented in Figure 5 and Figure 6. Specifically,

As shown in Figure 5, with increasing runtime, LN, Revive, and Shaduf exhibit a performance decline of 1% to 6% as the network gradually becomes imbalanced. In contrast, UCRb demonstrates significant improvement, particularly at lower capacity factors. For instance, at a capacity factor of 1, UCRb achieves a success ratio of 85%, which represents a 15% improvement over Horcrux at the same initial capacity. This success ratio continues to improve over time, eventually stabilizing at around 97%, reflecting an overall performance gain of 15%. Under long-term operation, UCRb demonstrates performance improvements in the range of 70%-74%, 42%-52%, 18%-30%, and 5%-15% when compared to LN, Revive, Shaduf, and Horcrux, respectively.

As shown in Figure 6, after prolonged operation, the number of depleted channels in the PCN increases by 2% and 8% for Revive

and Shaduf, respectively. In contrast, UCRb maintains a near-zero count of depleted channels even after long-term operation, similar to the behavior observed in Horcrux and LN.

**Incentive Distribution.** We have calculated the incentives for the nodes based on Algorithm **??**. Table 4. presents the incentive distribution for a few randomly selected transactions. The first column contains the set of nodes in transaction paths and corresponding coin shift information. The second column lists the incentives allocated to the nodes and their neighboring nodes involved in the coin shift process according to Case 1, while the third column shows the incentive distribution for the same nodes based on Case 2.

For the transaction path $\{1963, 9889, 8813, 500, 8900\}$, the corresponding coin shift information set is $\{\emptyset, \{\{1963, 206745\}\}, \emptyset, \{\{8813, 916455\}\}, \emptyset\}$. This indicates that node 1963 did not participate in any coin shift, while node 9889 performed a coin shift operation with node 1963 involving an amount of 206,745. Similarly, node 8813 performed a coin shift with another node involving an amount of 916,455, and the remaining nodes did not engage in any coin shift.

The incentives and losses of the nodes as per Case 1 are given by the set $\{-0.1, 0.0296, 0, 0.0703, 0\}$, where the sender incurs a loss of 0.1 as it provides incentives to the nodes that participated in the coin shift. Specifically, node 9889 receives an incentive of 0.0296 and node 500 receives 0.0703, reflecting a fair distribution based on each node's contribution to the total coin shift in the transaction.

In addition, the incentives distributed by the coin shift nodes to their respective neighbors are represented by the set $\{\emptyset, \{\{1963, 0.0148\}\}, \emptyset, \{\{8813, 0.0351\}\}, \emptyset\}$. We assume that each coin shift node transfers 50% of its received incentive to its neighbor(s), proportional to the neighbor's contribution to the total coin shift amount initiated by the node. Accordingly, node 9889 transfers an amount of 0.0148 to node 1963, and node 500 transfers 0.0351 to node 8813, ensuring a fair and proportional distribution of incentives.

The incentives and losses of nodes under Case 2 are represented by the set $\{0, 0.0388, -0.125, 0.0861, 0\}$, where intermediate nodes are responsible for providing incentives to the coin shift nodes. In this case, node 8813 gives an incentive of 0.0388 to node 9889 and 0.0861 to node 500, proportionally based on their respective contributions to the total coin shift that occurred in the transaction.

The incentives further distributed to the neighbors by the coin shift nodes are captured in the set $\{\emptyset, \{\{1963, 0.0194\}\}, \emptyset, \{\{8813, 0.043\}\}, \emptyset\}$. Here, node 9889 transfers 0.0194 to node 1963, and node 500 transfers 0.043 to node 8813. This is done according to the same principle of proportional contribution to the coin shift amount, ensuring fairness in the overall incentive distribution.

In summary, compared to LN [25], Revive [14], Shaduf [12], and Horcrux [27], UCRb demonstrates an improvement in success ratio ranging from 15%-50%, reduces user deposits by 72%-92%, and achieves an almost zero channel depletion rate. Furthermore, UCRb's long-term performance is approximately 1.5 times greater than its short-term performance, indicating that sustained operation is likely to yield even greater efficiency gains over time. We propose a novel incentive distribution scheme designed specifically for nodes that participate in the coin shift process.

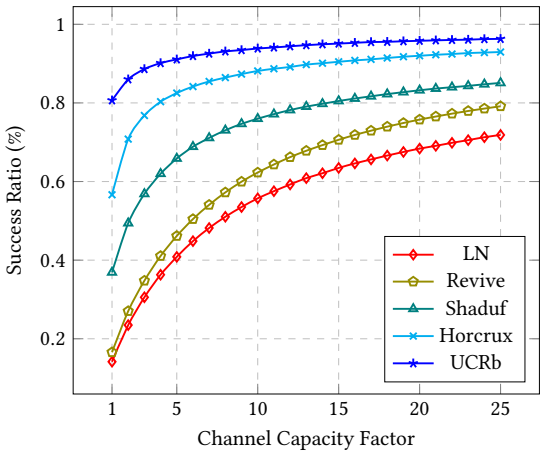



**Figure 3: Comparing the success ratio of 5 schemes under short-term operation, with running batches 20 and channel capacity factor from 1 to 25.**

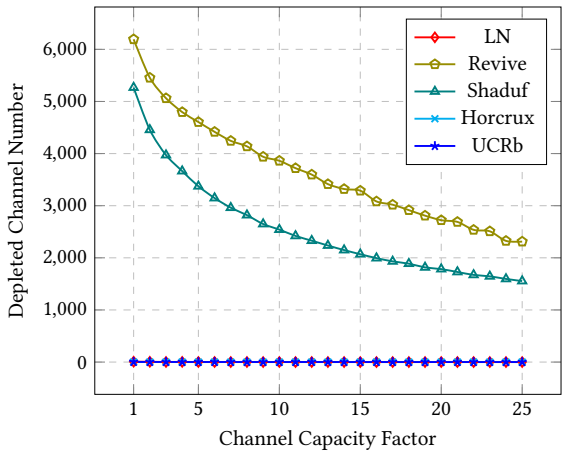



**Figure 4: Comparing the depleted channel number of 5 schemes under short-term operation, with running batches 20 and channel capacity factor from 1 to 25.**

## 7 Conclusion

We presented UCRb, a novel construction for PCNs that addresses the fundamental challenge of channel depletion through an adaptive, flexible, and reliable channel rebalancing mechanism. UCRb operates entirely off-chain, relying solely on digital signature verification and requiring no scripting or smart contract support. This ensures high compatibility with a wide range of blockchain platforms, including Bitcoin and Ethereum. To promote trustless cooperation, UCRb incorporates an incentive-compatible mechanism that motivates honest participation, even in adversarial and non-cooperative environments. It also introduces a new time-lock commitment scheme to guarantee atomic execution of payments, thereby preserving balance security. Additionally, UCRb enhances user privacy during off-chain operations through the use of zero-knowledge proofs. Experimental results demonstrate UCRb's superior efficiency, achieving near-zero channel depletion rates and significantly higher success rates compared to state-of-the-art solutions such as Horcrux and Shaduf. These outcomes establish UCRb as a

**Table 4: Incentive distribution for randomly selected 5 transactions under Case 1 and Case 2 scenarios**

| Trnasaction Path/Coin Shift | Case 1: Incentive to Nodes/ Neighbours | Case 2: Incentive to nodes/neighbours |
|---|---|---|
| {1963, 9889, 8813, 500, 8900}<br>Coin Shift per node:<br>{∅, {{1963, 206745}}, ∅,<br>{{8813, 916455}}, ∅} | {−0.1, 0.0296, 0, 0.0703, 0}<br>Incentive to neighbours:<br>{∅, {{1963, 0.0148}}, ∅,<br>{{8813, 0.0351}}, ∅} | {0, 0.0388, −0.125, 0.0861, 0}<br>Incentive to neighbours:<br>{∅, {{1963, 0.0194}}, ∅,<br>{{8813, 0.043}}, ∅} |
| {1791, 1392, 4515, 9826, 5097}<br>Coin Shift per node:<br>{∅, ∅, {{1392, 521615.5}},<br>{{4515, 275}, {9824, 43086}}, ∅} | {−0.1, 0, 0.0474, 0.0525, 0}<br>Incentive to neighbours:<br>{∅, ∅, {{1392, 0.0237}},<br>{{4515, 0.0001}, {9824, 0.0261}}, ∅} | {0, −0.1, 0.0602, 0.0397, 0}<br>Incentive to neighbours:<br>{∅, ∅, {{1392, 0.0301}},<br>{{4515, 0.0001}, {9824, 0.0197}}, ∅} |
| {2571, 1392, 7118, 2979, 2227}<br>Coin Shift per node:<br>{∅, ∅, ∅,<br>{{7118, 482500}}, ∅} | {−0.1, 0, 0, 0.1, 0}<br>Incentive to neighbours:<br>{∅, ∅, ∅,<br>{{7118, 0.05}}, 0} | {0, −0.1, −0.1, 0.2, 0}<br>Incentive to neighbours:<br>{∅, ∅, ∅,<br>{{7118, 0.1}}, ∅} |

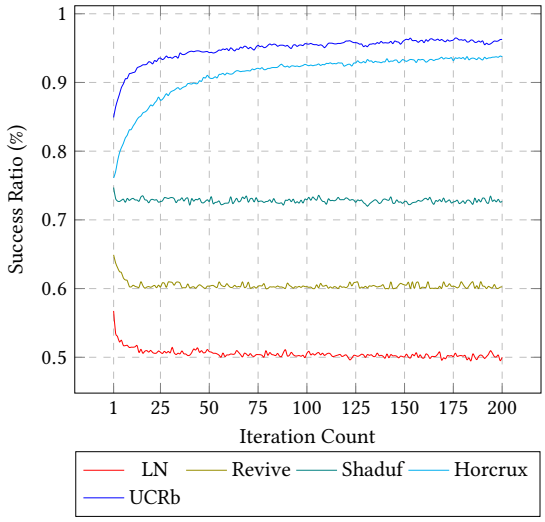



**Figure 5: Comparing the success ratio of 5 schemes under long-term operation, with channel capacity factor 8 and running batches from 1 to 200.**

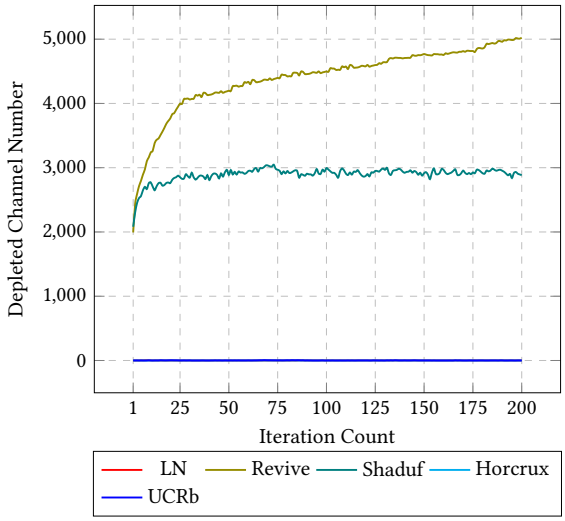



**Figure 6: Comparing the depleted channel number of 5 schemes under long-term operation, with channel capacity factor 8 and running batches from 1 to 200.**

scalable, cost-effective, and practical solution for long-term PCN sustainability. Future work includes formal security analysis under the Universal Composability (UC) framework and further optimizations, as discussed in Appendix L.

## Acknowledgments

The authors used ChatGPT4 only for grammatical revision of the text in the paper to correct any typos, grammatical errors, and awkward phrasing. This work was supported by the European Research Council (ERC) under the European Union's Horizon 2020 innovation program (grant PROCONTRA-885666).

## References

[1] Bitcoin Optech. Splicing, https://lightning.engineering/loop/.
[2] Lightning Labs. LOOP, https://bitcoinops.org/en/topics/splicing/.
[3] Raiden Network: Fast, cheap, scalable token transfers for Ethereum. https://raiden.network/, 2018.
[4] Lukas Aumayr, Kasra Abbaszadeh, and Matteo Maffei. Thora: Atomic and privacy-preserving multi-channel updates. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 165–178, 2022.
[5] Zeta Avarikioti, Krzysztof Pietrzak, Iosif Salem, Stefan Schmid, Samarth Tiwari, and Michelle Yeo. Hide & seek: Privacy-preserving rebalancing on payment channel networks. In *International Conference on Financial Cryptography and Data Security*, pages 358–373. Springer, 2022.
[6] Dan Boneh, Manu Drijvers, and Gregory Neven. Bls multi-signatures with public-key aggregation. *URL: https://crypto. stanford. edu/˜ dabo/pubs/papers/BLSmultisig. html*, 2018.
[7] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *International conference on the theory and application of cryptology and information security*, pages 514–532. Springer, 2001.

[8] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1084–1101. IEEE, 2019.
[9] S. Dziembowski, L. Eckey, S. Faust, and D. Malinowski. Perun: Virtual payment hubs over cryptocurrencies. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 106–123, May 2019.
[10] Christoph Egger, Pedro Moreno-Sanchez, and Matteo Maffei. Atomic multi-channel updates with constant collateral in bitcoin-compatible payment-channel networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 801–815, 2019.
[11] elizabethereum) Elizabeth (noot. Super cheap solidity schnorr sig verification using only ecrecover and keccak256. https://github.com/noot/schnorr-verify.
[12] Zhonghui Ge, Yi Zhang, Yu Long, and Dawu Gu. Shaduf: Non-cycle payment channel rebalancing. In *NDSS*, 2022.
[13] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ecdsa). *International journal of information security*, 1:36–63, 2001.
[14] Rami Khalil and Arthur Gervais. Revive: Rebalancing off-blockchain payment networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 439–453, 2017.
[15] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. Concurrency and privacy with payment-channel networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 455–471, 2017.
[16] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*, 2019.
[17] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, 2019.
[18] Susil Kumar Mohanty and Somanath Tripathy. n-htlc: Neo hashed time-lock commitment to defend against wormhole attack in payment channel networks. *Computers & Security*, 106:102291, 2021.
[19] Susil Kumar Mohanty and Somanath Tripathy. FlexiPCN: Flexible payment channel network. In *International Conference on Financial Cryptography and Data Security*, pages 405–419. Springer, 2023.
[20] Susil Kumar Mohanty and Somanath Tripathy. Flexichain: Flexible payment channel network to defend against channel exhaustion attack. *ACM Transactions on Privacy and Security*, 2024.
[21] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, http://bitcoin.org/bitcoin.pdf, 2008.
[22] Nicolas (Nixkitax). Zero Knowledge Proof for Schnorr in Circom. https://github.com/nixkitax/circom-schnorr-verify.
[23] Shimin Pan, Kwan Yin Chan, Handong Cui, and Tsz Hon Yuen. Multi-signatures for ecdsa and its applications in blockchain. In *Australasian Conference on Information Security and Privacy*, pages 265–285. Springer, 2022.
[24] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
[25] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. https://lightning.network/lightning-network-paper.pdf, 2016.
[26] Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of cryptology*, 4:161–174, 1991.
[27] Anqi Tian, Peifang Ni, Yingzi Gao, and Jing Xu. Horcrux: Synthesize, split, shift and stay alive preventing channel depletion via universal and enhanced multi-hop payments. *Cryptology ePrint Archive*, 2024.
[28] Samarth Tiwari, Michelle Yeo, Zeta Avarikioti, Iosif Salem, Krzysztof Pietrzak, and Stefan Schmid. Wiser: Increasing throughput in payment channel networks with transaction aggregation. In *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*, pages 217–231, 2022.
[29] Somanath Tripathy and Susil Kumar Mohanty. MAPPCN: Multi-hop anonymous and privacy-preserving payment channel network. In *International Conference on Financial Cryptography and Data Security*, pages 481–495. Springer, 2020.
[30] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger, https://ethereum.github.io/yellowpaper/paper.pdf, 2014.

# A PRELIMINARIES AND BACKGROUND

In this section, we provide the key preliminaries and cryptographic tools essential to our proposed work.

## A.1 Payment Channel Networks

Payment channels are off-chain constructs that allow two parties to perform a series of transactions by locking funds in a multi-signature address or a smart contract. These transactions are represented as a sequence of co-signed balance updates, requiring blockchain interaction only at the time of channel opening and closing. This design minimizes blockchain congestion and reduces both transaction fees and confirmation latency, thereby improving scalability [3, 25]. A Payment Channel Network (PCN) extends the bilateral channel model into a multi-hop architecture, enabling users without a direct channel to transact securely via a path of intermediaries. Each intermediary forwards the payment conditionally, under cryptographic guarantees commonly realized through Hashed Time-Lock Contracts (HTLCs), which ensure atomicity and prevent counterparty risk. This structure facilitates scalable, trustless micropayments across decentralized networks. Notable implementations include the Lightning Network for Bitcoin [25] and the Raiden Network for Ethereum [3].

## A.2 Hashed Time-Lock Contracts

The Hashed Timelock Contract (HTLC) serves as the fundamental atomic swap primitive in PCNs, enforcing conditional payment execution through cryptographic hash locks and temporal constraints. A sender creates a payment hash $H = \mathcal{H}(s)$ using secret $s$, which the recipient must reveal to claim funds within a predefined time window $\Delta t$. HTLCs guarantee atomicity across multi-hop routes, either all intermediate nodes receive their fees and the recipient obtains the payment, or the entire transaction fails and funds are refunded. However, HTLCs exhibit limitations including capital lockup during the timeout period and vulnerability to griefing attacks [10], wormhole attacks [16], channel exhaustion attacks [20].

## A.3 Channel Depletion in PCNs

While PCNs offer scalable off-chain transaction capabilities, they are susceptible to a phenomenon known as channel depletion or liquidity imbalance. This occurs when repeated unidirectional payments exhaust the available funds on one side of a payment channel, rendering it incapable of facilitating further transactions in that direction. Such imbalances can significantly degrade the network's efficiency, leading to increased payment failures and reduced overall throughput. To address channel depletion, various rebalancing strategies have been proposed. Traditional methods like Revive [14] utilize cycle-based rebalancing, requiring coordinated multi-party interactions, which may not be feasible in dynamic network conditions. Shaduf [12] introduces a non-cyclic off-chain rebalancing protocol, allowing users to shift funds between channels without forming cycles, thus offering greater flexibility and applicability in Ethereum blockchain platform. Building upon these approaches, Horcrux [27] presents a multi-party virtual channel based protocol that addresses channel depletion without relying on scripting languages and support for Bitcoin. Effectively mitigating channel depletion is crucial for the long-term viability of PCNs, especially in high-frequency transaction scenarios. Further research should focus on developing rebalancing protocols that balance efficiency, privacy, and decentralization while maintaining the atomicity and trustlessness inherent in off-chain payment systems.

## A.4 Pedersen Commitment

Pedersen Commitment [24] is a secure commitment scheme whose security is based on the discrete logarithm problem. The participants in this scheme primarily consist of a committer and a verifier, and the Pedersen Commitment scheme comprises the following steps:

- Setup: Let $\lambda$ be the security parameter. The generator samples the public parameter $p$ and $q$ to be two large primes such that $q$ divide $p - 1$. Then choose a generator $g$ of order $q$ subgroup of $Z_p^*$, and a random secret

$s \in Z_q$ that satisfies $h = g^s \ mod \ p$. Then the public parameters are $(p, q, g, h)$ based on the discrete logarithm problem.

- Commit: The committer creates the commitment of $x \in Z_q$ by choosing $r \in Z_q$ at random and computes the commitment $c = \text{com}(x, r) = g^x h^r \ mod \ p = g^x (g^s)^r \ mod \ p = g^{x+s \cdot r} mod \ p$, then $c$ will be sent to the verifier.
- Reveal: When the committer needs to open the commitment, it reveals the values of $x$ and $r$ to the verifier. The verifier then recalculates a commitment value $c' = \text{decom}(x, r) = g^x h^r \ mod \ p$ and verifies the authenticity of commitment $c$ by comparing $c$ with $c'$ to verify equality, i.e., $c \stackrel{?}{=} c'$.

## A.5 Multi-Signature

A multi-signature scheme allows a set of signers to individually generate a signature on a message $m$, which can then be combined to form a compact aggregated signature of all the signers on $m$, as well as combining the signers' public keys to form an aggregated public key for compact representation. We adhere to Drijvers et al.'s multisignature definition [8]. For public parameters $pp$, a multi-signature scheme (MuSig) consists of the following algorithms:

- $(\text{sk}, \text{pk}) \leftarrow \text{MuSig.KGen}(pp)$. On input the public parameter $pp$, the KGen algorithm outputs a public and secret key pair $(\text{pk}, \text{sk})$.
- $\text{apk} \leftarrow \text{MuSig.KAgg}(\vec{\mathbb{K}})$. On input a set of public key $\vec{\mathbb{K}} = \{pk_1, pk_2, \cdots, pk_n\}$, the KAgg algorithm outputs an aggregated public key apk.
- $\sigma_i \leftarrow \text{MuSig.Sign}(sk_i, m)$. On input of a secret key $\text{sk}_i$ and a message $m$, the Sign algorithm outputs a signature $\sigma_i$.
- $\sigma \leftarrow \text{MuSig.SigAgg}(m, \sigma_i)$. On input of a set of signatures $\sigma_i$, the SigAgg algorithm outputs a (succinct) aggregated signature $\sigma$.
- $0/1 \leftarrow \text{MuSig.Ver}(\text{apk}, m, \sigma)$. On input a message $m$, a signature $\sigma$, and an aggregated public key apk, the Ver algorithm returns 1 if the signature is valid; else, 0.

## A.6 Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Definition 1. *Let $\mathcal{R}$ be a binary relation for an NP language $L_{\mathcal{R}}$, where $\lambda$ is the security parameter. The argument system for $\mathcal{R}$ is defined as a quadruple probabilistic polynomial algorithms $\Pi = (\mathcal{G}, \mathcal{P}, \mathcal{V}, \mathcal{S})$ and a deterministic encoder $\mathcal{K}$, where:*

- *$pp \leftarrow \mathcal{G}(1^\lambda)$: The generator samples the public parameter $pp$ w.r.t. the security parameter $\lambda$.*
- *$(pk, vk) \leftarrow \mathcal{K}(pp, s)$: The prover and verifier key pair is derived from the commonly defined structure $s$ and the public parameter $pp$ using the deterministic encoder.*
- *$\pi \leftarrow \mathcal{P}(pk, u, w)$: Proving algorithm stating $(pp, s, u, w) \in \mathcal{R}$.*
- *$b \leftarrow \mathcal{V}(vk, u, \pi)$: Verification algorithm, where $b \in \{0, 1\}$.*
- *$\pi \leftarrow \mathcal{S}(pp, u, \tau)$: Simulator outputs $\pi$ given trapdoor $\tau$.*

A [zk]SNARKs is a non-interactive argument system with succinct proof size and verification time that satisfies the following properties:

- *Completeness:* If the statement is true, an honest verifier will be convinced by an honest prover.
- *Soundness:* A dishonest prover cannot convince an honest verifier of a false statement.
- *[Optional] Zero-Knowledge:* No additional information beyond the validity of the statement is revealed.

We formally define the properties of [zk]SNARKs in Appendix B.

# B Formal Definition of zkSNARKs

Formally, the properties of [zk]SNARKs are as follows.

Definition 2. *A non-interactive argument for $\mathcal{R}$ is a SNARK if it satisfies:*

- ***Completeness**: An honest prover with valid witness should convince any verifier. Formally, for any PPT adversary $\mathcal{A}$:*

$$Pr\left[\mathcal{V}(vk, u, \pi) = 1 \;\middle|\; \begin{array}{l} pp \leftarrow \mathcal{G}(1^\lambda) \\ (s, (u, w))) \leftarrow \mathcal{A}(pp) \\ (pp, s, u, w) \in \mathcal{R} \\ (pk, vk) \leftarrow \mathcal{K}(pp, s) \\ \pi \leftarrow \mathcal{P}(pk, u, w) \end{array}\right] = 1$$

- ***Knowledge Soundness**: A dishonest prover (adversary), should not be able to convince any verifier. To formally define this we require that for all PPT adversaries $\mathcal{A}$ there exists an extractor $\mathcal{E}$ that can compute witness given any randomness $\rho$, such that:*

$$\Pr\left[\begin{array}{l} \mathcal{V}(vk, u, \pi) = 1, \\ (pp, s, u, w) \notin \mathcal{R} \end{array} \;\middle|\; \begin{array}{c} pp \leftarrow \mathcal{G}(1^\lambda) \\ (s, (u, \pi)) \leftarrow \mathcal{A}(pp) \\ (pk, vk) \leftarrow \mathcal{K}(pp, s) \\ w \leftarrow \mathcal{E}(pp, \rho) \end{array}\right] = negl(\lambda)$$

- ***Zero-knowledge**: If the argument dos not reveal anything beyond the truth of the statement, we label it as zero-knowledge. Formally, there must exist a PPT simulator $\mathcal{S}$ such that for all PPT adversaries $\mathcal{A}$ following distributions are indistinguishable:*

$$\mathcal{D}_1 = \left\{ (pp, s, u, \pi) \;\middle|\; \begin{array}{l} pp \leftarrow \mathcal{G}(1^\lambda) \\ (s, (u, w))) \leftarrow \mathcal{A}(pp) \\ (pp, s, u, w) \in \mathcal{R} \\ (pk, vk) \leftarrow \mathcal{K}(pp, s) \\ \pi \leftarrow \mathcal{P}(pk, u, w) \end{array} \right\}$$

$$\mathcal{D}_2 = \left\{ (pp, s, u, \pi) \;\middle|\; \begin{array}{l} (pp, \rho) \leftarrow \mathcal{S}(1^\lambda) \\ (s, (u, w))) \leftarrow \mathcal{A}(pp) \\ (pp, s, u, w) \in \mathcal{R} \\ (pk, vk) \leftarrow \mathcal{K}(pp, s) \\ \pi \leftarrow \mathcal{S}(pp, u, \rho) \end{array} \right\}$$

# C Discussion

## C.1 Payment Channel Sustainability

When a payment channel becomes depleted, it typically cannot support further off-chain transactions, disrupting the payment network. The user can dynamically shift coins from one or more neighboring channels, depending on availability, into the depleted channel. This capability helps to replenish funds and restore functionality without relying on on-chain operations. The core innovation lies in the flexibility of the coin-shifting mechanism, which does not require closing or reopening channels or interacting with the blockchain. Instead, when a coin shift is initiated among three users, they first agree on a common state that reflects the updated balances. This agreement is recorded in the form of a coin shift proof, which ensures transparency and verifiability for all users in the network. Since this proof can be independently validated by the other users, it maintains security and consistency across the network. As a result, channels remain usable over time without depleting, and the overall network stays resilient and cost-efficient by minimizing expensive on-chain interactions.

## C.2 Universal Blockchain Compatibility

The proposed rebalancing mechanism is designed to be blockchain-agnostic, meaning it can be deployed on any blockchain platform that supports standard verifiable digital signature schemes such as Schnorr [26], ECDSA [13], or BLS [7]. This flexibility arises from the fact that nearly all operations—including the construction and verification of proofs such as the coin allocation proof and coin shift proof—are performed entirely off-chain. The only on-chain requirement is the verification of a multisignature [6, 17, 23], a basic cryptographic primitive already supported by modern blockchain systems like Bitcoin [21] and Ethereum [30]. Because the protocol's core logic and security assurances are enforced off-chain, it does not rely on any specific blockchain architecture, smart contract platform, or consensus mechanism. As a result, our rebalancing mechanism offers high portability, scalability, and ease of integration across a diverse range of blockchain environments, without requiring changes to the underlying infrastructure.

# D Assumptions

We assume that every algorithm implicitly takes the blockchain as input, which is publicly accessible to all users. We assume a synchronous communication model in which each user responds within a predefined time. Each pair of users uses a secure and authenticated channel to exchange payment related information. While the sender and receiver share a secure channel, they do not have such channels with intermediate nodes. Each intermediary is only aware of its immediate neighbors both the previous and the next user in the path. The sender has detailed knowledge of the network topology, which includes the identities of participating users, locktimes, relay fees and channel rebalancing fees for each intermediate node, channel identifiers, and node capacities, though not the capacities of the individual channels. For a transaction to proceed, at least one valid payment path must exist between the sender and receiver, with all involved channels satisfying the collateral requirements either through direct funding or by shifting coins from other channels.

# E Adversary Model

We consider a *rational adversary* $\mathcal{A}$, whose objective is to maximize its own profit. In addition, $\mathcal{A}$ is modeled as a *static adversary*, meaning it selects a subset of users from the set $\mathcal{U} = \{u_1, \ldots, u_n\}$ to fully corrupt before the protocol execution begins. The choice of corrupted users remains fixed throughout the protocol. Crucially, the adversary ensures that at least one user in $\{\mathcal{P}_1, \ldots, \mathcal{P}_n\}$ remains honest.

## E.1 Security and Privacy Goals

Motivated by prior works [12, 20, 27], this work aims to achieve two fundamental objectives: *balance security*, which ensures that honest users do not incur any financial losses, *value privacy*, which ensures that the precise allocation of coins across multiple channels remains confidential during coin allocation, and *user anonymity*, which prevents the disclosure of participants' identities during coin shift operations. The core security and privacy properties addressed by our protocol are outlined below:

- **Consensus on Coin Deposit:** A coin deposit is considered valid only if all neighboring users agree to it. When all neighboring users are honest, the consensus required for coin deposit is achieved in constant time. However, it takes a delay of $O(\Delta)$ time once the request is initiated.
- **Consensus on Coin Allocation:** A coin allocation is accepted as valid only when consent is obtained from all neighbors. If all such users act honestly, the consensus can be reached in constant time.
- **Consensus on Coin Shift:** A coin shift is considered valid only when it receives mutual approval from at least three parties: the two neighboring users and a common participant shared by both adjacent channels. If all three users act honestly, consensus can be achieved in constant time.
- **Consensus on Coin Transfer:** A coin transfer and state transitions of a channel require mutual consent from both users involved in the payment channels. Consensus is achieved in constant time when both users are honest.
- **Guaranteed Coin Settlement:** A coin settlement is valid only with agreement from all neighboring users. Any user can request a settlement at any time. When all neighbors are honest, consensus is reached in constant time, and the settlement is completed in $O(\Delta)$ time once initiated.
- **Guaranteed Coin Allocation Privacy:** It guarantees that, during the coin allocation phase, the exact values assigned to individual channels remain indistinguishable to external observers. This confidentiality protects against adversarial inference of internal fund partitioning strategies and mitigates the balance inference attack.
- **Guaranteed Coin Shift Privacy:** It guarantees that, during coin transfers across adjacent payment channels, the identity of the intermediary user facilitating the shift remains confidential. This property ensures unlinkability between the source and destination channels, thereby preserving the user anonymity and preventing adversarial correlation or deanonymization of users.

# F Coin Allocation Proof

## F.1 Coin Allocation Statement

This proof is used to prove the coin allocation operation among the neighboring nodes of the user $u_i$. informally, the user $u_i$ wants to prove that the summation of all the coin allocation to the neighboring nodes is equal to the total on-chain deposit of the user $u_i$.

**Proof**: $\pi^{ca}_{u_i} \leftarrow \mathsf{P}_{\mathsf{NIZK}}\big(\langle v_{u_i}, \Delta_{u_i}\rangle, \mathcal{N}(u_i), (\Upsilon_{ik})_{u_k \in \mathcal{N}(u_i)}\big)$

- $\langle v_{u_i}, \Delta_{u_i}\rangle$: Public inputs of the proof
- $\mathcal{N}(u_i)$: Set of all neighboring users of $u_i$
- $\Delta_{u_i}$: a list of commitments to all the $n$ neighbors of $u_i$, i.e., $\Delta_{u_i} = [\delta_{i,0}, \ldots, \delta_{i,n-1}]$.
- $\Upsilon_{ik}$: The channel balance from $u_i$ to $u_k$ for $u_k \in \mathcal{N}(u_i)$.

Definition 3. *Assume that there is a coin allocation operation between the user $u_i$ and its neighbors $u_k \in \mathcal{N}(u_i)$ with a list of commitments $\Delta_{u_i} = [\delta_{i,0}, \ldots, \delta_{i,n-1}]$ to all the neighbors of $u_i$. The user $u_i$ wants to prove that the summation of all the coin allocation to the neighboring nodes is equal to their total on-chain deposit $v_{u_i}$. The statement is formally written as:*

$$S[v_{u_i}, \Delta_{u_i}] : \Big\{\forall u_k \in \mathcal{N}(u_i),\ \exists\, \Upsilon_{ik}, r_{ik}, c_{\langle u_i, u_k\rangle}\ s.t.$$
$$\sum_{k=0}^{n} \Upsilon_{ik} = v_{u_i} \wedge \forall k \in [0, n-1],\ \delta_{i,k} = H(\Upsilon_{ik} \| c_{\langle u_i, u_k\rangle} \| r_{ik})\Big\}$$

Theorem 1. *The statement in Definition 3 satisfies the soundness property of the general zkSNARKs scheme in the PPT adversarial model, while preserving the confidentiality of both the coin allocation amount and neighboring nodes' identities.*

Proof Sketch. To break the soundness of the above statement, an adversary must succeed in one of the following two cases: (1) The adversary breaks the soundness of the underlying zkSNARK scheme—i.e., they generate a valid proof without knowledge of a correct witness. In this context, it would imply that $\sum_{k=0}^{n} \Upsilon_{ik} \neq v_{u_i}$, yet the proof is still accepted. This violates the soundness property of zkSNARKs and is assumed to be infeasible for any PPT adversary. (2) The adversary finds a collision in the underlying hash function $H$ when computing the value $\delta_{i,k} = H(\Upsilon_{ik} \| c_{\langle u_i, u_k\rangle} \| r_{ik})$. Producing such a collision would contradict the collision resistance of $H$, which again is assumed to be secure against any PPT adversary. □

# G Coin Shift Proof

## G.1 Coin Shift Statement

This proof is used to prove the coin shift operation by user $u_i$ between its channels with two users $u_j$ and $u_k$.

**Proof**: $\pi^i_{kj} \leftarrow \mathsf{P}_{\mathsf{NIZK}}(\langle \mathsf{cs}, f_{u_i}, l_{u_i}, h_{u_i}, v^i_{kj}, v_{ij}\rangle, c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, \Upsilon'_{ik}, \complement'_{ik}, \Upsilon'_{ij}, \complement'_{ij}, \sigma^i_{ik}, \sigma^i_{ij}, \sigma^k_{ik}, \sigma^j_{ij})$

- $\langle \mathsf{cs}, f_{u_i}, l_{u_i}, h_{u_i}, v^i_{kj}, v_{ij}\rangle$: Public inputs of the proof
- cs: Coinshift number
- $f_{u_i}$: Processing charge of $u_i$
- $l_{u_i}$: Rebalancing fee lower rate of $u_i$
- $h_{u_i}$: Rebalancing fee higher rate of $u_i$
- $v^i_{kj}$: Amount of coin shift is done by $u_i$ from channel $c_{\langle u_i,u_k\rangle}$ to channel $c_{\langle u_i,u_j\rangle}$
- $v_{ij}$: Total amount $u_i$ wanted/ required to rebalance or coin shift from other neighboring users $\mathcal{N}(u_i)$
- $\mathcal{N}(u_i)$: Set of all neighbouring users of $u_i$
- $c_{\langle u_i,u_k\rangle}$: Payment channel identifier between $u_i$ and $u_k$
- $c_{\langle u_i,u_j\rangle}$: Payment channel identifier between $u_i$ and $u_j$
- $\mathcal{P}\{u_1 \rightarrow u_2 \rightarrow \cdots \rightarrow u_n\}$: Payment path or PCN
- $\Upsilon_{ij}$: Collateral or channel balance from $u_i$ to $u_j$ (i.e., $u_i \rightarrow u_j$)
- $\Upsilon_{ji}$: Collateral or channel balance from $u_j$ to $u_i$ (i.e., $u_j \rightarrow u_i$)
- $\complement_{ij}$ or $\complement_{ji}$: Capacity of channel $c_{\langle u_i,u_j\rangle}$ (same for $c_{\langle u_j,u_i\rangle}$)
- $\Upsilon'_{ik}$: The channel balance is from $u_i$ to $u_k$, however the prime symbol is used here to indicate the user $u_k$, who may or may not be part of the payment path from which the user $u_i$ is moving coin. The prime symbol will only be used for CoinShift operations. Similar for the case of channel capacity $\complement'_{ik}$.
- $\sigma^i_{ij}$: The signature is signed by user $u_i$ for the channel $c_{\langle u_i,u_j\rangle}$.
- $\sigma^j_{ij}$: The signature is signed by user $u_j$ for the channel $c_{\langle u_i,u_j\rangle}$.

Definition 4. *Assume that there is a coin shift happening between three nodes $u_j, u_m, u_k$, as depicted in Fig. 2. In this scenario, $u_i$ as the initiator is performing the coin shift operation transferring $v^i_{kj}$ amount of coin from channel with $u_k$ to the channel with $u_j$. After finalizing the operation by all three involved parties according to Algorithm 3 and Fig. 2, $u_i$ generates a zkSNARKs proof for the following statement:*

$$
\begin{aligned}
S[\mathsf{cs}, f_{u_i}, l_{u_i}, h_{u_i}, v^i_{kj}, v_{ij}] : \\
\Big\{ \exists\, \sigma^i_{ik}, \sigma^i_{ij}, \sigma^k_{ik}, \sigma^j_{ij}, \Upsilon_{ik}, \Upsilon_{ij}, \complement_{ij}, \complement_{ik}, v^i_{kj} s.t. \\
\Upsilon'_{ik} = \Upsilon_{ik} - v^i_{kj} \wedge \complement'_{ik} = \Upsilon'_{ik} + \Upsilon_{ik} \\
\wedge\, \Upsilon'_{ij} = \Upsilon_{ij} + v^i_{kj} \wedge \complement'_{ij} = \Upsilon'_{ij} + \Upsilon_{ik} \\
m_{ij} = c_{\langle u_i,u_j\rangle} \| v^i_{kj} \wedge m_{ik} = c_{\langle u_i,u_k\rangle} \| v^i_{kj} \\
\wedge\, \mathcal{V}_\sigma(m_{ij}, \sigma^j_{ij}, pk_j) = 1 \wedge \mathcal{V}_\sigma(m_{ij}, \sigma^i_{ij}, pk_i) = 1 \\
\wedge\, \mathcal{V}_\sigma(m_{ik}, \sigma^k_{ik}, pk_k) = 1 \wedge \mathcal{V}_\sigma(m_{cs}, \sigma^i_{ik}, pk_i) = 1 \Big\}
\end{aligned}
$$

Theorem 2. *The statement in Definition 4 satisfies the soundness property of the general zkSNARKs scheme in the PPT adversarial model.*

Proof sketch. In order to break the soundness of the above statement, the adversary needs to either (1) break the soundness of the underlying zkSNARKs scheme, meaning that the adversary can geenrate a valid proof without possession of the complete witness: $\sigma^i_{ik}, \sigma^i_{ij}, \sigma^k_{ik}, \sigma^j_{ij}, \Upsilon_{ik}, \Upsilon_{ij}, \complement_{ij}, \complement_{ik}, v^i_{kj}$. Or (2) break the security of the digital signature scheme, or (3) the adversary could have found a collision in the underlying hash function before signing the coin-shift agreement messages $m_{ij}, m_{ik}, m_{cs}$. □

# H Universal Channel Rebalancing (UCRb) Protocol

We present the concrete pseudocode for the Universal Channel Rebalancing (UCRb) protocol in Algorithm 1.

# I Off-chain Payment Protocol based on Pedersen Commitment-based Time Lock Contract (PCTLC)

We present the concrete pseudocode for the Off-chain Payment Protocol based on Pedersen Commitment-based Time Lock Contract (PCTLC) in Algorithm 2.

# J Off-chain CoinShift Operation

We present the concrete pseudocode for the Off-chain CoinShift Operation in Algorithm 3.

# K Incentive Mechanism for CoinShift Operation

We present the concrete pseudocode for the Incentive Mechanism for CoinShift Operation in Algorithm **??**.

# L Future Work

This work establishes a comprehensive framework for secure and efficient off-chain payment channel networks; however, several critical research directions for further research. Strengthening the protocol's security through formal analysis in the Universal Composability (UC) framework would provide robust guarantees against a wide range of adversarial behaviors. Enhancing on-chain dispute resolution mechanisms, particularly under adversarial conditions, is essential for improving the reliability and trustworthiness of the system. Extending the protocol to support generic coin shift operations for non-transactional rebalancing would significantly increase its flexibility and applicability in dynamic network conditions. A key challenge also lies in the development of a concrete and provably optimal reward mechanism to incentivize intermediary nodes for active participation in cooperative coin shift processes. While the current design employs a heuristic-based incentive scheme, a formally grounded and incentive-compatible model remains an open research problem. Improving the efficiency of coin allocation during rebalancing involves developing adaptive strategies that consider real-time channel states and network conditions. Similarly, determining the optimal neighbor from which to source coins during rebalancing demands a careful balance between liquidity utilization and operational efficiency. Optimizing payment path selection is equally critical; identifying paths that minimize transaction cost and latency while maximizing liquidity availability remains an open problem. Advancing these directions will enhance the scalability, robustness, and usability of the proposed off-chain payment framework, moving it closer to practical deployment in real-world decentralized financial ecosystems.

**Algorithm 1** Universal Channel Rebalancing (UCRb) Protocol

Assume $p$ and $q$ are two large primes, such that $q|p-1$, $g$ is a generator of the order $q$ subgroup of $\mathbb{Z}_q^*$

**CoinDeposit**

**Initiator $u_i$'s CoinDeposit Routine:**

**CoinDeposit**$(u_i, \mathcal{N}(u_i), v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, t_{u_i})$

**Set**: $\mathsf{m}_{u_i} \leftarrow \{v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, t_{u_i}\}$
**Signature**: $\sigma_{u_i} \leftarrow \mathsf{Sign}(\mathsf{sk}_{u_i}, \mathsf{m}_{u_i})$
**Copy**: $\mathcal{N}'(u_i) \leftarrow \mathcal{N}(u_i)$
**while** $\mathcal{N}'(u_i) \neq \emptyset$ **do**
  **Extract**: $u_j \leftarrow \mathsf{extract}(\mathcal{N}(u_i))$
  **if** $(\{u_i, u_j\} \in \mathcal{B})$ and $(c_{\langle u_i,u_j\rangle} \notin \mathcal{OC})$ **then**
    **Create**: Channel identifier $c_{\langle u_i,u_j\rangle}$
    **Set**: $\Upsilon_{ij} \leftarrow 0, \Upsilon_{ji} \leftarrow 0, \complement_{ij} \leftarrow 0$
    **Send**: $(\mathsf{open}, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_i}, \sigma_{u_i})$ to $u_j$
    **Receive**: $(\top, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_j}, \sigma_{u_j})$ from $u_j$
    **Insert**: $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \mathsf{m}_{u_i}, \mathsf{m}_{u_j}, \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{OC}$
  **else**
    **Abort**
  **end if**
  **Difference**: $\mathcal{N}'(u_i) \leftarrow \mathcal{N}'(u_i) \setminus \{u_j\}$
**end while**
**MultiSig**: $\sigma \leftarrow \mathsf{SigAgg}(\mathsf{m}_{u_i}, \sigma_{u_i}), \forall u_i \in \mathcal{N}(u_i) \cup \{u_i\}$
**Write**: $\mathsf{CoinDeposit}(u_i, \mathcal{N}(u_i), \mathsf{m}_{u_i}, \sigma)$ to $\mathcal{B}$

**Nighbor $u_j$'s CoinDeposit Routine:**

**Receive**: $(\mathsf{open}, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_i}, \sigma_{u_i})$ from $u_i$
$\mathsf{m}_{u_i} : \{v_{u_i}, f_{u_i}, l_{u_i}, h_{u_i}, t_{u_i}\}$
**if** $(\mathcal{B}[u_i] \cdot \mathsf{bal} \geq v_{u_i})$ and $(t_{u_i} > \mathcal{B}[t_{now}])$ **then**
  **Set**: $\Upsilon_{ij} \leftarrow 0, \Upsilon_{ji} \leftarrow 0, \complement_{ij} = 0, t_{u_j} \leftarrow t_{u_i}$
  **Set**: $\mathsf{m}_{u_j} \leftarrow \{v_{u_j}, f_{u_j}, l_{u_j}, h_{u_j}, t_{u_j}\}$
  **Signature**: $\sigma_{u_j} \leftarrow \mathsf{Sign}(\mathsf{sk}_{u_j}, \mathsf{m}_{u_i})$
  **Insert**: $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \mathsf{m}_{u_i}, \mathsf{m}_{u_j}, \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{OC}$
  **Send**: $(\top, c_{\langle u_i,u_j\rangle}, \mathsf{m}_{u_j}, \sigma_{u_j})$ to $u_i$
**else**
  **Abort**
**end if**

**CoinAllocation**

**Initiator $u_i$'s CoinAllocation Routine:**

**CoinAllocation**$(u_i, \mathcal{N}(u_i), v_{u_i})$

**Initialise**: $n \leftarrow |\mathcal{N}(u_i)|$ ▷ # of neighbouring user of $u_i$
**for** $j = 1$ to $n$ **do**
  $\Upsilon_{ij} \leftarrow v_{u_i}/n$ ▷ Not limited to this allocation policy
**end for**
**Proof**: $\pi_{u_i}^{ca} \leftarrow \mathsf{P}_{\mathsf{NIZK}}(v_{u_i} = \sum_{j=1}^{n} \Upsilon_{ij})$
**for** $j = 1$ to $n$ **do**
  **if** $(c_{\langle u_i,u_j\rangle} \in \mathcal{OC})$ **then**
    **Send**: $(c_{\langle u_i,u_j\rangle}, \pi_{u_i}^{ca}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i})$ to $u_j$
    **Receive**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$ from $u_j$
    **Update**: $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \pi_{u_i}^{ca}, \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{OC}$
  **else**
    **Abort**
  **end if**
**end for**
**MultiSig**: $\sigma \leftarrow \mathsf{SigAgg}(\pi_{u_i}^{ca}, \sigma_{u_i}) \ \forall u_i \in \mathcal{N}(u_i) \cup \{u_i\}$
**Insert**: $(u_i, \mathcal{N}(u_i), \Upsilon, \pi_{u_i}^{ca}, \sigma)$, in $\mathcal{OC}$ ▷ $\Upsilon$: Coin allocation set

**Neighbor $u_j$'s CoinAllocation Routine:**

**Receive**: $(c_{\langle u_i,u_j\rangle}, \pi_{u_i}^{ca}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i})$ from $u_i$
**Verify**: $\mathsf{b} \leftarrow \mathsf{V}_{\mathsf{NIZK}}(\mathsf{stmt}, \pi_{u_i}^{ca})$
**if** $(b \stackrel{?}{=} 1)$ and $(c_{\langle u_i,u_j\rangle} \in \mathcal{OC})$ **then**
  **Update**: $(c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \pi_{u_i}^{ca}, \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{OC}$
  **Send**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$ to $u_i$
**else**
  **Abort**
**end if**

**Payment**

**Call:** Algorithm 2, Off-chain payment protocol.

**CoinShift**

**Call:** Algorithm 3, Off-chain coinshift operation.

**CoinSettlement**

**Initiator $u_i$'s CoinSettlement Routine:**

**CoinSettlement**$(u_i, \mathsf{F}_{\mathsf{state}}, \sigma)$

**Initialise**: $n \leftarrow |\mathcal{N}(u_i)|$ ▷ # of neighbouring user of $u_i$
**Set**: $\mathsf{F}_{\mathsf{state}} \leftarrow \{\mathsf{state}(c_{\langle u_i,u_1\rangle}) \ || \ \cdots \ || \ \mathsf{state}(c_{\langle u_i,u_n\rangle})\}$
  ▷ $\mathsf{state}(c_{\langle u_i,u_j\rangle}) : (c_{\langle u_i,u_j\rangle}, \Upsilon_{ij}, \Upsilon_{ji}, \complement_{ij}, \sigma_{u_i}, \sigma_{u_j})$
**Signature**: $\sigma_{u_i} \leftarrow \mathsf{Sign}(\mathsf{sk}_{u_i}, \mathsf{F}_{\mathsf{state}})$
**for** $j = 1$ to $n$ **do**
  **Send**: $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \sigma_{u_i})$ to $u_j$
  **Receive**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$ from $u_j$
  **Insert**: $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \mathsf{state}(c_{\langle u_i,u_j\rangle}), \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{CC}$
**end for**
**MultiSig**: $\sigma \leftarrow \mathsf{SigAgg}(\mathsf{F}_{\mathsf{state}}, \sigma_{\mathsf{u_i}}) \ \forall u_i \in \mathcal{N}(u_i) \cup \{u_i\}$
**Write** $\mathsf{CoinSettlement}(u_i, \mathsf{F}_{\mathsf{state}}, \sigma)$ to $\mathcal{B}$

**Neighbor $u_j$'s CoinSettlement Routine:**

**Receive**: $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \sigma_{u_i})$ from $u_i$
**if** $(\mathsf{F}_{\mathsf{state}}[\mathsf{state}(c_{\langle u_i,u_j\rangle})] \stackrel{?}{=} u_j \cdot \mathcal{OC}[\mathsf{state}(c_{\langle u_i,u_j\rangle})])$ **then**
  **Insert**: $(c_{\langle u_i,u_j\rangle}, \mathsf{F}_{\mathsf{state}}, \mathsf{state}(c_{\langle u_i,u_j\rangle}), \sigma_{u_i}, \sigma_{u_j})$ in $\mathcal{CC}$
  **Send**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{u_j})$ to $u_i$
**else**
  **Abort**
**end if**

**Algorithm 2** Off-chain Payment Protocol based on Pedersen Commitment-based Time Lock Contract (PCTLC)

**Sender $u_0$'s Setup Phase:**

**Generate**: $\{s, r\} \xleftarrow{\$} \mathbb{Z}_q$
**Compute**: $\kappa \leftarrow g^s \pmod p$
**Send**: $(\kappa, r, v, \delta)$ to $u_n$ via a secured channel

**Sender $u_0$'s Payment Routine:**

**Compute**: $v_{01} \leftarrow v + \sum_{i=1}^{n-1} f_{u_i} + \delta$ ▷ Extra amount
**if** $(v_{01} \leq \Upsilon_{01})$ **then** ▷ Locking/ Commit Phase
  **Compute**: $\Upsilon_{01} \leftarrow \Upsilon_{01} - v_{01}$
  **Compute**: $\Upsilon_{10} \leftarrow \Upsilon_{10} + v_{10}$
  **Compute**: $t_{01} \leftarrow t_{now} + \Delta \cdot n - 1$
  **Assign**: $\pi_{01}^{cs} \leftarrow \emptyset$ ▷ $\emptyset$: Empty set
  **Generate**: $\alpha_{01} \xleftarrow{\$} \mathbb{Z}_q$
  **Compute**: $\beta_{01} \leftarrow g^{\alpha_{01}} \cdot \kappa^r \pmod p$
  **Compute**: $\gamma_{01} \leftarrow g^{\alpha_{01}} \pmod p$
  $\mathsf{PCTLC}(u_0, u_1, v_{01}, t_{01}, \beta_{01}, \gamma_{01}, \pi_{01}^{cs}, \sigma_{01})$
**else**
  **Abort**
**end if**
**Receive**: $(\varphi, \vartheta_{10}, \pi^{cs}, v_{10}, \sigma_{10})$ from $u_1$
**Call**: $(\mathrm{a} \leftarrow \mathsf{calculateAmount}(\delta, \mathrm{pi}, \pi^{cs}, \mathrm{pi}_{10}, \pi_{10}^{cs}))$ ▷ Unlocking/ Reveal Phase
**Verify**: $\mathrm{b} \leftarrow \mathsf{V}_{\mathsf{NIZK}}(\mathrm{stmt}, \pi^{cs})$
**if** $(\mathrm{a} \stackrel{?}{=} v_{10})$ and $(\mathrm{b} \stackrel{?}{=} 1)$ and $(\vartheta_{10} \stackrel{?}{=} \varphi \cdot \beta_{01} \pmod p)$ and $(\kappa \stackrel{?}{=} \vartheta_{10} \cdot g^{-\alpha_{01}} \pmod p)$ **then**
  **if** $(v_{01} \geq v_{10})$ **then**
    **Update**: $\Upsilon_{01} \leftarrow \Upsilon_{01} + (v_{01} - v_{10})$
    **Update**: $\Upsilon_{10} \leftarrow \Upsilon_{10} - (v_{01} - v_{10})$
    **Update**: $\complement_{01} \leftarrow \Upsilon_{01} + \Upsilon_{10}$
    **Update**: $(c_{\langle u_0, u_1 \rangle}, \Upsilon_{01}, \Upsilon_{10}, \complement_{01}, \pi^{cs}, \sigma_{01}, \sigma_{10})$ in $OC$
  **else**
    **Update**: $\Upsilon_{01} \leftarrow \Upsilon_{01} + v_{01}$
    **Update**: $\Upsilon_{10} \leftarrow \Upsilon_{10} - v_{01}$
    **Abort**
  **end if**
**else**
  **Update**: $\Upsilon_{01} \leftarrow \Upsilon_{01} + v_{01}$
  **Update**: $\Upsilon_{10} \leftarrow \Upsilon_{10} - v_{01}$
  **Abort**
**end if**

**Intermediary $u_i$'s Payment Routine:**

**Compute**: $v_{\langle i,i+1 \rangle} \leftarrow v_{\langle i-1,i \rangle} - f_{u_i}$ ▷ Locking Phase
**if** $(v_{\langle i,i+1 \rangle} \leq \Upsilon_{\langle i,i+1 \rangle})$ and $(t_{\langle i,i+1 \rangle} \stackrel{?}{=} t_{\langle i-1,i \rangle} - \Delta)$ **then**
  **Compute**: $t_{\langle i,i+1 \rangle} \leftarrow t_{\langle i-1,i \rangle} - \Delta$
  **Compute**: $\Upsilon_{\langle i,i+1 \rangle} \leftarrow \Upsilon_{\langle i,i+1 \rangle} - v_{\langle i,i+1 \rangle}$
  **Compute**: $\Upsilon_{\langle i+1,i \rangle} \leftarrow \Upsilon_{\langle i+1,i \rangle} + v_{\langle i,i+1 \rangle}$
  **Generate**: $\alpha_{\langle i,i+1 \rangle} \xleftarrow{\$} \mathbb{Z}_q$
  **Compute**: $\beta_{\langle i,i+1 \rangle} \leftarrow g^{\alpha_{\langle i,i+1 \rangle}} \cdot \beta_{\langle i-1,i \rangle} \pmod p$
  **Compute**: $\gamma_{\langle i,i+1 \rangle} \leftarrow g^{\alpha_{\langle i,i+1 \rangle}} \cdot \gamma_{\langle i-1,i \rangle} \pmod p$
  **if** $v_{\langle i,i+1 \rangle} = 0$ **then Assign**: $\pi_{\langle i,i+1 \rangle}^{cs} \leftarrow \pi_{\langle i-1,i \rangle}^{cs} \cup \emptyset$ **Assign**: $\mathrm{pi}_{\langle i,i+1 \rangle}^{cs} \leftarrow \mathrm{pi}_{\langle i-1,i \rangle}^{cs} \cup \mathrm{pi}_{\langle i,i+1 \rangle}$
  **end if**
**else**
  **Call**: $\pi_{\langle i,i+1 \rangle}^{cs} \leftarrow \pi_{\langle i-1,i \rangle}^{cs} \cup \mathsf{CoinShift}(u_i, \mathcal{N}u_i), c_{\langle i,i+1 \rangle}, v_{\langle i,i+1 \rangle})$ ▷ Neighboring set $\mathcal{N}(u_i) = \{u_j \in V | (u_i, u_j) \in E\}$
  **Goto:** Step 2
**end if**
$\mathsf{BPTLC}(u_i, u_{i+1}, v_{\langle i,i+1 \rangle}, t_{\langle i,i+1 \rangle}, \beta_{\langle i,i+1 \rangle}, \gamma_{\langle i,i+1 \rangle}, \pi_{\langle i,i+1 \rangle}^{cs}, \sigma_{\langle i,i+1 \rangle})$
**Receive**: $(\varphi, \vartheta_{\langle i+1,i \rangle}, \pi^{cs}, v_{\langle i+1,i \rangle}, \sigma_{\langle i+1,i \rangle})$ from $u_{i+1}$
**Call**: $\mathrm{a} \leftarrow \mathsf{calculateAmount}(\delta, \mathrm{pi}, \pi^{cs}, \mathrm{pi}_{\langle i,i+1 \rangle}, \pi_{\langle i,i+1 \rangle}^{cs})$ ▷ Unlocking and Reveal Phase
**Verify**: $\mathrm{b} \leftarrow \mathsf{V}_{\mathsf{NIZK}}(\mathrm{stmt}, \pi^{cs})$
**if** $(\mathrm{a} \stackrel{?}{=} v_{\langle i+1,i \rangle})$ and $(\mathrm{b} \stackrel{?}{=} 1)$ and $(\vartheta_{\langle i+1,i \rangle} \stackrel{?}{=} \varphi \cdot \beta_{\langle i,i+1 \rangle} \pmod p)$ **then**
  **if** $(v_{\langle i,i+1 \rangle} \geq v_{\langle i+1,i \rangle})$ **then**
    **Update**: $\Upsilon_{\langle i,i+1 \rangle} \leftarrow \Upsilon_{\langle i,i+1 \rangle} + (v_{\langle i,i+1 \rangle} - v_{\langle i+1,i \rangle})$
    **Update**: $\Upsilon_{\langle i+1,i \rangle} \leftarrow \Upsilon_{\langle i+1,i \rangle} - (v_{\langle i,i+1 \rangle} - v_{\langle i+1,i \rangle})$
    **Update**: $\complement_{\langle i,i+1 \rangle} \leftarrow \Upsilon_{\langle i,i+1 \rangle} + \Upsilon_{\langle i+1,i \rangle}$
    **Compute**: $\vartheta_{\langle i,i-1 \rangle} \leftarrow \vartheta_{\langle i+1,i \rangle} \cdot g^{-\alpha_{\langle i,i+1 \rangle}} \pmod p$
    **Call**: $v_{\langle i,i-1 \rangle} \leftarrow \mathsf{calculateAmount}(\delta, \mathrm{pi}, \pi^{cs}, \mathrm{pi}_{\langle i-1,i \rangle}, \pi_{\langle i-1,i \rangle}^{cs})$
    **Update**: $\Upsilon_{\langle i-1,i \rangle} \leftarrow \Upsilon_{\langle i-1,i \rangle} - \left(v_{\langle i-1,i \rangle} - v_{\langle i,i-1 \rangle}\right)$
    **Update**: $\Upsilon_{\langle i,i-1 \rangle} \leftarrow \Upsilon_{\langle i,i-1 \rangle} + \left(v_{\langle i-1,i \rangle} - v_{\langle i,i-1 \rangle}\right)$.
    **Update**: $\complement_{\langle i-1,i \rangle} \leftarrow \Upsilon_{\langle i-1,i \rangle} + \Upsilon_{\langle i,i-1 \rangle}$
    **Update**: $(c_{\langle i,i+1 \rangle}, \Upsilon_{\langle i,i+1 \rangle}, \Upsilon_{\langle i+1,i \rangle}, \complement_{\langle i,i+1 \rangle}, \pi^{cs}, \sigma_{\langle i,i+1 \rangle}, \sigma_{\langle i+1,i \rangle})$ in $OC$
    **Update**: $(c_{\langle i,i-1 \rangle}, \Upsilon_{\langle i,i-1 \rangle}, \Upsilon_{\langle i-1,i \rangle}, \complement_{\langle i-1,i \rangle}, \pi^{cs}, \sigma_{\langle i-1,i \rangle}, \sigma_{\langle i,i-1 \rangle})$ in $OC$
    **Send**: $(\varphi, \vartheta_{\langle i,i-1 \rangle}, v_{\langle i,i-1 \rangle}, \sigma_{\langle i,i-1 \rangle})$ to $u_{i-1}$
  **else**
    **Update**: $\Upsilon_{\langle i,i+1 \rangle} \leftarrow \Upsilon_{\langle i,i+1 \rangle} + v_{\langle i,i+1 \rangle}$
    **Update**: $\Upsilon_{\langle i+1,i \rangle} \leftarrow \Upsilon_{\langle i+1,i \rangle} - v_{\langle i,i+1 \rangle}$
    **Abort**
  **end if**
**else**
  **Update**: $\Upsilon_{\langle i,i+1 \rangle} \leftarrow \Upsilon_{\langle i,i+1 \rangle} + v_{\langle i,i+1 \rangle}$
  **Update**: $\Upsilon_{\langle i+1,i \rangle} \leftarrow \Upsilon_{\langle i+1,i \rangle} - v_{\langle i,i+1 \rangle}$
  **Abort**
**end if**

**Receiver $u_n$'s Payment Routine:**

**Receive**: $(c_{\langle u_{n-1}, u_n \rangle}, v_{\langle n-1,u_n \rangle}, t_{\langle n-1,u_n \rangle}, \beta_{\langle n-1,u_n \rangle}, \gamma_{\langle n-1,u_n \rangle}, \pi_{\langle n-1,n \rangle}^{cs}, \sigma_{\langle n-1,n \rangle})$ from $u_{n-1}$
**Verify**: $\mathrm{b} \leftarrow \mathsf{V}_{\mathsf{NIZK}}(\mathrm{stmt}, \pi_{\langle n-1,n \rangle}^{cs})$
**if** $(v \stackrel{?}{=} v_{\langle n-1,n \rangle})$ and $(\mathrm{b} \stackrel{?}{=} 1)$ and $(\beta_{\langle n-1,n \rangle} \stackrel{?}{=} \gamma_{\langle n-1,n \rangle} \cdot \kappa^r \pmod p)$ and $(t_{\langle n-1,n \rangle} \stackrel{?}{=} t_{now} + \Delta)$ **then** ▷ Reveal Phase
  **Compute**: $\varphi \leftarrow \kappa \cdot \kappa^{-r} \pmod p$
  **Compute**: $\vartheta_{\langle n,n-1 \rangle} \leftarrow \kappa \cdot \gamma_{\langle n-1,n \rangle} \pmod p$
  **Assign**: $v_{\langle n,n-1 \rangle} \leftarrow v$
  **Assign**: $\pi^{cs} \leftarrow \pi_{\langle n,n-1 \rangle}^{cs}$
  **Update**: $\Upsilon_{\langle n-1,n \rangle} \leftarrow \Upsilon_{\langle n-1,n \rangle} - v_{\langle n-1,n \rangle}$
  **Update**: $\Upsilon_{\langle n,n-1 \rangle} \leftarrow \Upsilon_{\langle n,n-1 \rangle} + v_{\langle n-1,n \rangle}$
  **Update**: $\complement_{\langle n-1,n \rangle} \leftarrow \Upsilon_{\langle n-1,n \rangle} + \Upsilon_{\langle n,n-1 \rangle}$
  **Update**: $(c_{\langle n-1,n \rangle}, \Upsilon_{\langle n-1,n \rangle}, \Upsilon_{\langle n,n-1 \rangle}, \complement_{\langle n-1,n \rangle}, \pi^{cs}, \sigma_{\langle n-1,n \rangle}, \sigma_{\langle n,n-1 \rangle})$ in $OC$
  **Send**: $(\varphi, \vartheta_{\langle n,n-1 \rangle}, v_{\langle n,n-1 \rangle}, \sigma_{\langle n,n-1 \rangle})$ to $u_{n-1}$
**else**
  **Abort**
**end if**

**Algorithm 3** Off-chain CoinShift Operation

Assume $p$ and $q$ are two large primes, such that $q|p-1$, $g$ is a generator of the order $q$ subgroup of $\mathbb{Z}_q^*$

**Initiator $u_i$'s CoinShift Routine:**
**CoinShift**$(u_i, \mathcal{N}(u_i), c_{\langle u_i,u_j\rangle}, v_{ij})$

**Set**: cs $\leftarrow 0$, $\pi_{u_i}^{cs} \leftarrow \emptyset$ ▷ cs: Coin shift number
**Choose**: a neighbor(s) $N^{'}(u_i) \subseteq \mathcal{N}(u_i)$, such that $\sum_{u_k \in N^{'}(u_i)} \Upsilon_{ik} == v_{ij}$
**Set**: $\beta_{\text{previous}} \leftarrow \beta_{hi}$, $\gamma_{\text{previous}} \leftarrow \gamma_{hi}$
**while** $v_{ij} \neq 0$ **do** ▷ Locking/ Commit Phase
  **if** $(u_k \in N^{'}(u_i))$ and $(c_{\langle u_i,u_k\rangle} \in OC)$ **then**
    **Generate**: $\alpha'_{ik} \xleftarrow{\$} \mathbb{Z}_q$
    **Compute**: $\beta'_{ik} \leftarrow g^{\alpha'_{ik}} \cdot \beta_{\text{previous}} (\text{mod } p)$
    **Compute**: $\gamma'_{ik} \leftarrow g^{\alpha'_{ik}} \cdot \gamma_{\text{previous}} (\text{mod } p)$
    **Send**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \beta'_{ik}, \gamma'_{ik}, \sigma_{ik}^i, \sigma_{ij}^i)$ to $u_k$
    **Send**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ij}^i)$ to $u_j$
    **Receive**: $(\top, c_{\langle u_i,u_k\rangle}, \beta'_{ki}, \gamma'_{ki}, \sigma_{ik}^k, \sigma_{ij}^j)$ from $u_k$
    **Receive**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{ij}^j, \sigma_{ik}^k)$ from $u_j$
    **Compute**: $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - v_{kj}^i$
    **Compute**: $\complement'_{ik} \leftarrow \Upsilon'_{ik} + \Upsilon_{ki}$
    **Update**: $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon_{ki}, \complement'_{ik}, \sigma_{ik}^i, \sigma_{ik}^k)$ in $OC$
    **Compute**: $\Upsilon'_{ij} \leftarrow \Upsilon_{ij} + v_{kj}^i$
    **Compute**: $\complement'_{ij} \leftarrow \Upsilon'_{ij} + \Upsilon_{ji}$
    **Update**: $(c_{\langle u_i,u_j\rangle}, \Upsilon'_{ij}, \Upsilon_{ji}, \complement'_{ij}, \sigma_{ij}^i, \sigma_{ij}^j)$ in $OC$
    **Update**: cs $\leftarrow$ cs + 1
    **Proof**: $\pi_{kj}^i \leftarrow \mathsf{P}_{\mathsf{NIZK}}(\text{pi}_{kj}^i, w)$
      ▷ $\text{pi}_{kj}^i : \langle \text{cs}, f_{u_i}, l_{u_i}, h_{u_i}, v_{kj}^i, v_{ij}\rangle$
      $w : (c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, \Upsilon'_{ik}, \complement'_{ik}, \Upsilon'_{ij}, \complement'_{ij}, \sigma_{ik}^i, \sigma_{ij}^i, \sigma_{ik}^k, \sigma_{ij}^j)$
    **Update**: $\pi_{u_i}^{cs} \leftarrow \pi_{u_i}^{cs} \cup \pi_{kj}^i$
    **Insert**: $(\text{cs}, \pi_{u_i}^{cs}, c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, \Upsilon'_{ik}, \complement'_{ki}, \Upsilon'_{ij}, \complement'_{ij}, \sigma_{ik}^i, \sigma_{ik}^k, \sigma_{ij}^i, \sigma_{ij}^j)$ in $\mathcal{CS}$ ▷ $\mathcal{CS}$: Coin Shift list
  **end if**
  **Assign:** $\beta_{\text{previous}} \leftarrow \beta'_{ik}$, $\gamma_{\text{previous}} \leftarrow \gamma'_{ik}$
  **Extract**: $u_k \leftarrow \text{next}(u_k, N^{'}(u_i))$
  **Compute**: $v_{ij} \leftarrow v_{ij} - v_{kj}^i$
**end while**
**Receive**: $(c_{\langle u_i,u_j\rangle}, \varphi, \vartheta_{ji}, v_{ji}, \sigma_{ji})$ from $u_j$
**while** $N^{'}(u_i) \neq \emptyset$ **do** ▷ Unlocking/ Reveal Phase
  **Extract**: $u_k \leftarrow \text{extract}(N^{'}(u_i))$
  **Compute**: $\vartheta_{ik}^{'} \leftarrow \vartheta_{ji} \cdot g^{-\alpha_{ij}} \ (\text{mod } p)$
  **Call**: $\ell_{ik}^{'} \leftarrow \text{getIncentiveNeighbour}(\text{pi}_{kj}^i, \pi_{kj}^i)$
  **Send**: $(c_{\langle u_i,u_k\rangle}, \varphi, \vartheta_{ik}^{'}, \ell_{ik}^{'}, \pi^{cs}, \sigma_{ik}^i)$ to $u_k$
  **Receive**: $(\top, \vartheta_{ki}^{'}, \sigma_{ik}^k)$ from $u_k$
  **if** $(\vartheta_{ki}^{'} \stackrel{?}{=} \varphi \cdot \beta'_{ik} \ (\text{mod } p))$ **then**
    **Compute**: $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - \ell'_{ik}$
    **Compute**: $\Upsilon'_{ki} \leftarrow \Upsilon_{ki} + \ell'_{ik}$
    **Update**: $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon'_{ki}, \complement'_{ik}, \pi^{cs}, \sigma_{ik}^i, \sigma_{ik}^k)$ in $OC$
    **Difference**: $N^{'}(u_i) \leftarrow N^{'}(u_i) \setminus \{u_k\}$
  **else**
    **Abort**
  **end if**
**end while**

**Source Neighbor $u_k$'s CoinShift Routine:**

**Receive**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ij}^i)$ from $u_i$
**if** $(\Upsilon_{ik} \geq v_{kj}^i)$ **then** ▷ Locking/ Commit Phase
  **Send**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ik}^k)$ to $u_j$
  **Receive**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ij}^i, \sigma_{ij}^j)$ from $u_j$
  **Compute**: $\Upsilon_{ik}^{'} \leftarrow \Upsilon_{ik} - v_{kj}^i$
  **Compute**: $\complement_{ik}^{'} \leftarrow \Upsilon_{ik}^{'} + \Upsilon_{ki}$
  **Generate:** $\alpha'_{ki} \xleftarrow{\$} \mathbb{Z}_q$
  **Compute:** $\beta'_{ki} \leftarrow g^{\alpha'_{ki}} \cdot \beta'_{ik} (\text{mod } p)$
  **Compute:** $\gamma'_{ki} \leftarrow g^{\alpha'_{ki}} \cdot \gamma'_{ik} (\text{mod } p)$
  **Update**: $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon_{ki}, \complement_{ik}^{'}, \sigma_{ik}^i, \sigma_{ik}^k)$ in $OC$
  **Insert**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ik}^k, \sigma_{ij}^i, \sigma_{ij}^j)$ in $\mathcal{CS}$
  **Send**: $(\top, c_{\langle u_i,u_k\rangle}, \beta'_{ki}, \gamma'_{ki}, \sigma_{ik}^k, \sigma_{ij}^j)$ to $u_i$
**end if**
**Receive**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, \varphi, \vartheta_{ik}^{'}, \ell_{ik}^{'}, \pi^{cs}, \sigma_{ik}^i)$
**Verify**: b $\leftarrow \mathsf{V}_{\mathsf{NIZK}}(\text{stmt}, \pi_{kj}^i)$ ▷ Unlocking/ Reveal Phase
**if** $(b \stackrel{?}{=} 1)$ and $(\vartheta_{ik}^{'} \stackrel{?}{=} \varphi \cdot \beta'_{ki} (\text{mod } p))$ and $(\ell'_{ik} \stackrel{?}{=} \text{getIncentiveNeighbour}(\text{pi}_{kj}^i, \pi_{kj}^i))$ **then**
  **Compute**: $\Upsilon'_{ik} \leftarrow \Upsilon_{ik} - \ell_{ik}^{'}$
  **Compute**: $\Upsilon'_{ki} \leftarrow \Upsilon_{ki} + \ell_{ik}^{'}$
  **Update**: $(c_{\langle u_i,u_k\rangle}, \Upsilon'_{ik}, \Upsilon'_{ki}, \complement_{ik}^{'}, \pi_{kj}^i, \sigma_{ik}^i, \sigma_{ik}^k)$ in $OC$
  **Send**: $(\top, c_{\langle u_i,u_k\rangle}, \varphi, \vartheta_{ki}^{'}, \sigma_{ik}^k)$ to $u_i$
**end if**

**Target Neighbor $u_j$'s CoinShift Routine:**

**Receive**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ij}^i)$ from $u_j$
**if** $(\mathcal{B}[u_i] \geq v_{kj}^i)$ **then**
  **Send**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ij}^i, \sigma_{ij}^j)$ to $u_k$
  **Receive**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ik}^i, \sigma_{ik}^k)$ from $u_k$
  **Compute**: $\Upsilon'_{ij} \leftarrow \Upsilon_{ij} - v_{kj}^i$
  **Compute**: $\complement'_{ij} \leftarrow \Upsilon'_{ij} + v_{kj}^i$
  **Update**: $(c_{\langle u_i,u_j\rangle}, \Upsilon'_{ij}, \Upsilon_{ji}, \complement'_{ij}, \sigma_{ij}^i, \sigma_{ij}^j)$ in $OC$
  **Insert**: $(c_{\langle u_i,u_k\rangle}, c_{\langle u_i,u_j\rangle}, v_{kj}^i, \sigma_{ij}^i, \sigma_{ij}^j, \sigma_{ik}^i, \sigma_{ik}^k)$ in $\mathcal{CS}$
  **Send**: $(\top, c_{\langle u_i,u_j\rangle}, \sigma_{ij}^j, \sigma_{ik}^k)$ to $u_i$
**end if**